



# HealthSync

## Hospital Management System

### OBJECT-ORIENTED ANALYSIS & DESIGN REPORT

UE23CS352B: Object Oriented Analysis and Design

| Name         | SRN           |
|--------------|---------------|
| Yuti Naha    | PES1UG23CS723 |
| Yuvika T     | PES1UG23CS725 |
| Vaishnavi JP | PES1UG23CS900 |
| Prerana M P  | PES1UG23CS903 |

| Technology      | Java 17 + Spring Boot 3.x + H2 Database           |
|-----------------|---------------------------------------------------|
| Architecture    | MVC (Model–View–Controller) with REST API         |
| Design Patterns | Factory Method · Singleton · Facade · Observer    |
| Frontend        | Single-Page Application (HTML / CSS / JavaScript) |

## Section 1 — Detailed Synopsis

---

### 1.1 Project Background & Problem Statement

In a bustling healthcare environment, manual record-keeping leads to data silos, scheduling conflicts, and delayed patient care. Traditional hospitals rely on fragmented, paper-based processes where patient history, doctor schedules, billing, and pharmacy records are maintained in isolation. This results in scheduling conflicts when two patients book the same doctor slot, billing errors due to manual aggregation of charges, loss of medical history when patients move between departments, and lack of real-time visibility into ward occupancy or pharmacy stock.

HealthSync is designed to automate the clinical and administrative workflows of a medium-sized hospital. The system facilitates seamless interaction between Patients, Doctors, Nurses/Staff, and Administrators to ensure that patient history, appointment scheduling, and billing are handled with high precision and zero data loss.

### 1.2 System Vision

HealthSync serves as an integrated digital solution for modern healthcare facilities. It provides a centralised, role-aware platform where every actor can perform their work without duplication of effort, reducing errors and improving patient outcomes. The system consolidates patient records, doctor availability, appointment scheduling, billing, pharmacy inventory, and lab test management into a single unified web application.

### 1.3 Highlight Features

#### Major Features

- **Patient Registration & Profiling** — The system shall allow new patients to register and create a unique Patient ID for tracking all future medical interactions.
- **Doctor Availability & Appointment** — **Scheduling:** The system must verify doctor availability in real-time before confirming a booking to prevent scheduling conflicts.
- **Digital Medical Records (EMR)** — Doctors must be able to create, update, and search for patient records by ID, Name, or Contact Number for clinical diagnosis.
- **Billing & Insurance** — Calculate total costs based on consultation fees, medications, and insurance coverage to generate a final invoice and process insurance claims.

#### Minor Features

- **User Authentication & Role-Based Access** — Secure login with specific access levels for Patients, Doctors, Staff, and Administrators enforced via role guards.
- **Automated Pharmacy Inventory Management** — Staff can manage medicine stocks with automatic status updates after every sale or issuance.
- **E-Prescription Generation** — Digital prescriptions are transmitted directly to the pharmacy for immediate processing after a doctor writes them.
- **Lab Report Notifications** — Automated alerts sent to patients when their diagnostic results are ready using the Observer pattern.
- **Search & Filter** — Patients can find available doctors based on specialization or department.

### 1.4 Functional Requirements

| ID    | Requirement                                                                                                                                                          | Priority |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-01 | Patient Registration & Profiling: Allow new patients to register and create a unique Patient ID for tracking all future medical interactions.                        | Major    |
| FR-02 | Doctor Availability & Appointment Scheduling: Verify doctor availability in real-time before confirming a booking to prevent scheduling conflicts.                   | Major    |
| FR-03 | Digital Medical Records (EMR): Doctors must be able to create, update, and search for patient records by ID, Name, or Contact Number for clinical diagnosis.         | Major    |
| FR-04 | Billing & Insurance: Calculate total costs based on consultation fees, medications, and insurance coverage to generate a final invoice and process insurance claims. | Major    |
| FR-05 | Role-Based Authentication: Support different user types (Patients, Doctors, Staff) with specific access levels for security.                                         | Minor    |
| FR-06 | Lab Report Notifications: Generate and send alerts to patients when their diagnostic results are ready for viewing.                                                  | Minor    |
| FR-07 | Automated Pharmacy Inventory: Staff can manage medicine stocks, with the system automatically updating status after every sale or issuance.                          | Minor    |
| FR-08 | Search & Filter: Patients can search for available doctors using specialization or department as criteria.                                                           | Minor    |

### 1.5 Non-Functional Requirements

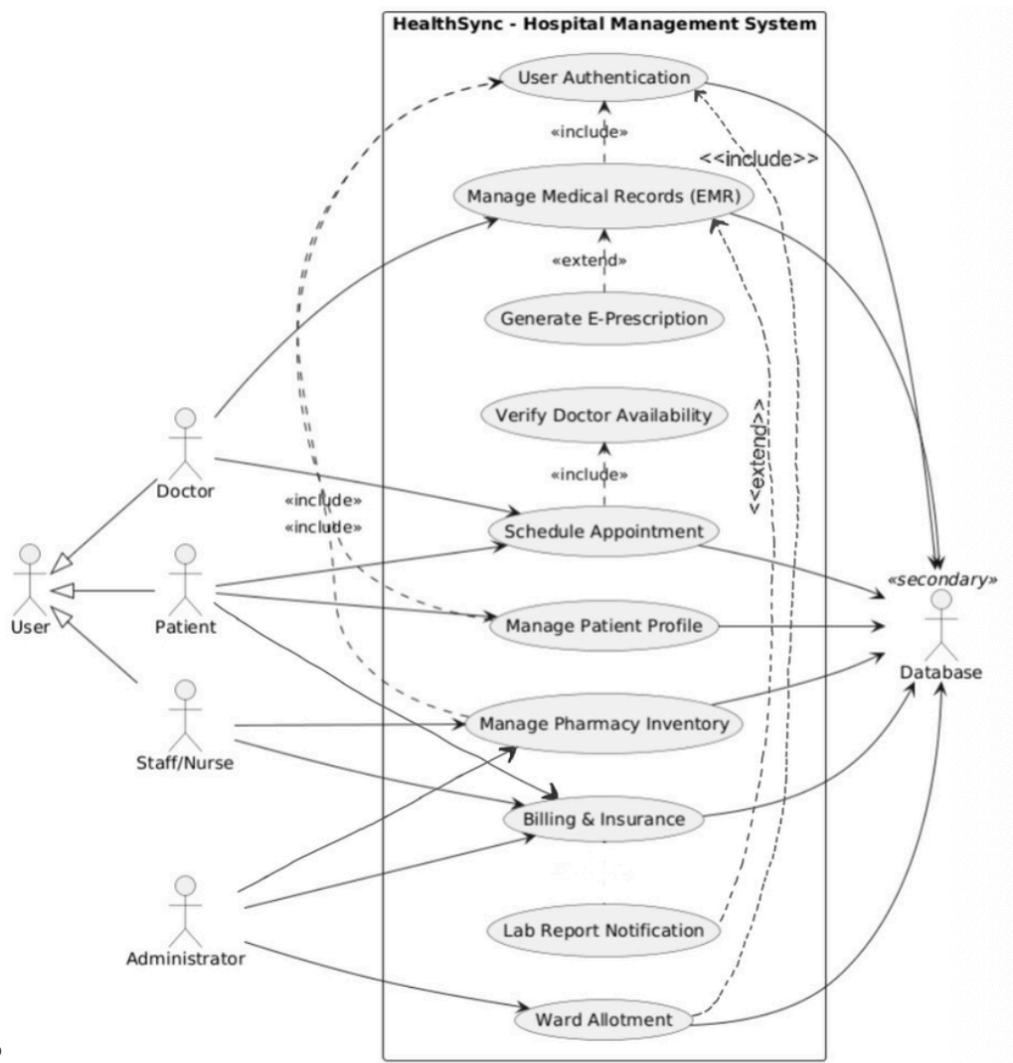
| ID     | Requirement                                                                                                          | Priority |
|--------|----------------------------------------------------------------------------------------------------------------------|----------|
| NFR-01 | Data Security: All sensitive medical history must be encrypted, and data access restricted to authorized roles only. | Major    |
| NFR-02 | Data Reliability: All data must be persisted in a database to ensure zero data loss during system failures.          | Major    |
| NFR-03 | System Scalability: Handle a minimum of 100 concurrent users without performance degradation.                        | Major    |
| NFR-04 | High Availability: Application must be accessible 24/7 for emergency record retrieval.                               | Major    |
| NFR-05 | UI Consistency: Clean, consistent interface with white background screens for reports.                               | Minor    |
| NFR-06 | Audit Logging: Log every transaction and user action for administrative monitoring.                                  | Minor    |
| NFR-07 | Response Time: Process patient search queries and display results within 2 seconds.                                  | Minor    |
| NFR-08 | Platform Compatibility: Run seamlessly on any standard operating system as a web application.                        | Minor    |

### 1.6 Technology Stack

| Layer           | Technology                                                                           |
|-----------------|--------------------------------------------------------------------------------------|
| Backend         | Java 17 + Spring Boot 3.x                                                            |
| Persistence     | Spring Data JPA + H2 In-Memory Database                                              |
| REST API        | @RestController with JSON responses (no server-side rendering)                       |
| Frontend (View) | HTML5 / CSS3 / Vanilla JavaScript — Single-Page Application (index.html)             |
| Build Tool      | Apache Maven                                                                         |
| Patterns        | Factory Method, Singleton (Creational) · Facade (Structural) · Observer (Behavioral) |
| Security        | SHA-256 password hashing + X-User-Role header guard (RoleGuard)                      |

## Section 2 — Use Case Model

The Use Case diagram shows the HealthSync system boundary with 4 primary actors and 9 use cases connected via «include» and «extend» relationships.



## 2.1 Actors

| Actor         | Type                      | Responsibilities                                                                                    |
|---------------|---------------------------|-----------------------------------------------------------------------------------------------------|
| User          | Abstract / Generalization | The base actor. Patient, Doctor, Staff/Nurse, and Administrator all generalize from User.           |
| Patient       | Primary                   | Self-registers, books appointments, views medical records, pays bills, receives notifications.      |
| Doctor        | Primary                   | Views schedule, conducts consultations, creates EMRs, orders lab tests, writes prescriptions.       |
| Staff / Nurse | Primary                   | Manages pharmacy inventory, generates invoices, records payments, allots wards, conducts lab tests. |
| Administrator | Primary                   | Manages all users, views audit logs, manages departments, oversees all modules.                     |
| Database      | Secondary                 | Stores all persistent data — acts as a secondary actor in the use case diagram.                     |

## 2.2 Use Case Descriptions

The following four use cases are the primary (High priority) use cases implemented in HealthSync, matching FR-01 through FR-04:

### UC1 — Patient Registration & Profile Management (FR-01)

Precondition: Patient has not previously registered in the system.

Main Flow:

1. Patient navigates to the registration page.
2. Patient enters: full name, email, phone, password, blood group, emergency contact, date of birth.
3. System validates all fields (checks for duplicate email).
4. System generates a unique Patient ID (format: PAT00001) via `HealthSyncFactory.createPatient()`.
5. System creates a linked User entity (USR00001) via `HealthSyncFactory.createUser()`.
6. Database stores both User and Patient records.
7. System creates the patient profile and sends a confirmation.
8. Patient views confirmation and is redirected to login.

Alternate Flow: If email already exists → Display error 'Email already registered' → Patient re-enters details.

Postcondition: Patient has a unique Patient ID and can log into the system.

Includes: «include» User Authentication

Class Diagram elements used: User, Patient, HealthSyncFactory, AuditLog

### UC2 — Appointment Scheduling (FR-02)

Precondition: Patient is registered and logged in. Doctor has available slots.

Main Flow:

1. Patient logs into HealthSync.
2. Patient searches for a doctor by specialization or department.
3. System retrieves list of matching doctors from the Department.
4. Patient selects a doctor.
5. System checks doctor availability in real-time (`DoctorAvailabilitySlot`).
- 6a. Slots available → System displays available time slots.

- 6b. No slots → Patient added to waitlist; notified when slot opens.
7. Patient selects date and time slot.
8. System verifies no scheduling conflict (slot.isBooked check).
9. Patient confirms appointment.
10. System generates Appointment ID, stores record, marks slot as booked.
11. Doctor's schedule is updated. Patient receives confirmation notification (Observer).

Alternate Flow: Conflict detected → Notify slot unavailable → Patient re-selects slot.

Postcondition: Appointment is SCHEDULED; slot is marked booked; patient notified.

Includes: «include» Verify Doctor Availability; «extend» Schedule Appointment

Class Diagram elements: Patient, Doctor, Appointment, DoctorAvailabilitySlot, AppointmentService, Notification

### UC3 — Manage Medical Records (EMR) (FR-03)

Precondition: Doctor is authenticated. Patient has a registered profile.

Main Flow:

1. Patient requests consultation.
2. Doctor logs in and authenticates.
3. Doctor searches for patient record by ID, name, or contact.
- 4a. Record exists → Doctor opens existing record.
- 4b. No record → Doctor creates a new MedicalRecord (status = OPEN).
5. Patient provides medical history.
6. Doctor saves new record to database.
7. Doctor adds diagnosis notes.
- 8a. No lab test required → Doctor proceeds to write prescription.
- 8b. Lab test required → Staff/Nurse conducts lab test and uploads results.
9. Doctor reviews lab results.
10. Doctor writes prescription; sent to pharmacy (PrescriptionStatus = PENDING).
11. Doctor saves and closes the record (status = CLOSED).
12. Database updates medical record; Patient receives notification.

Postcondition: MedicalRecord is CLOSED with diagnosis, lab results, prescription attached.

Extends: «extend» Generate E-Prescription

Class Diagram elements: MedicalRecord, Doctor, Patient, Prescription, LabTest, Notification

### UC4 — Billing & Insurance (FR-04)

Precondition: Patient has received consultation. Services are recorded.

Main Flow:

1. Staff retrieves patient record.
2. Staff selects services used during visit.
3. Billing System calculates: consultation fee + medication cost + tax.
4. System generates invoice (BillingInsurance object created, status = InvoiceGenerated).
5. Patient receives invoice notification.
6. Patient chooses payment method: Insurance or Direct Payment.
- 6a. Direct Payment → Patient makes direct payment → Billing System processes it.
- 6b. Insurance → Patient submits insurance claim → System processes claim → Applies coverage.
7. Payment successful → status = PAID → System sends receipt.
- 7b. Payment failed → status = PENDING → Patient retries.

Alternate: cancel / failure → status = CANCELLED.

Postcondition: Invoice has status PAID or PARTIALLY\_PAID or PENDING. Receipt sent via Observer.

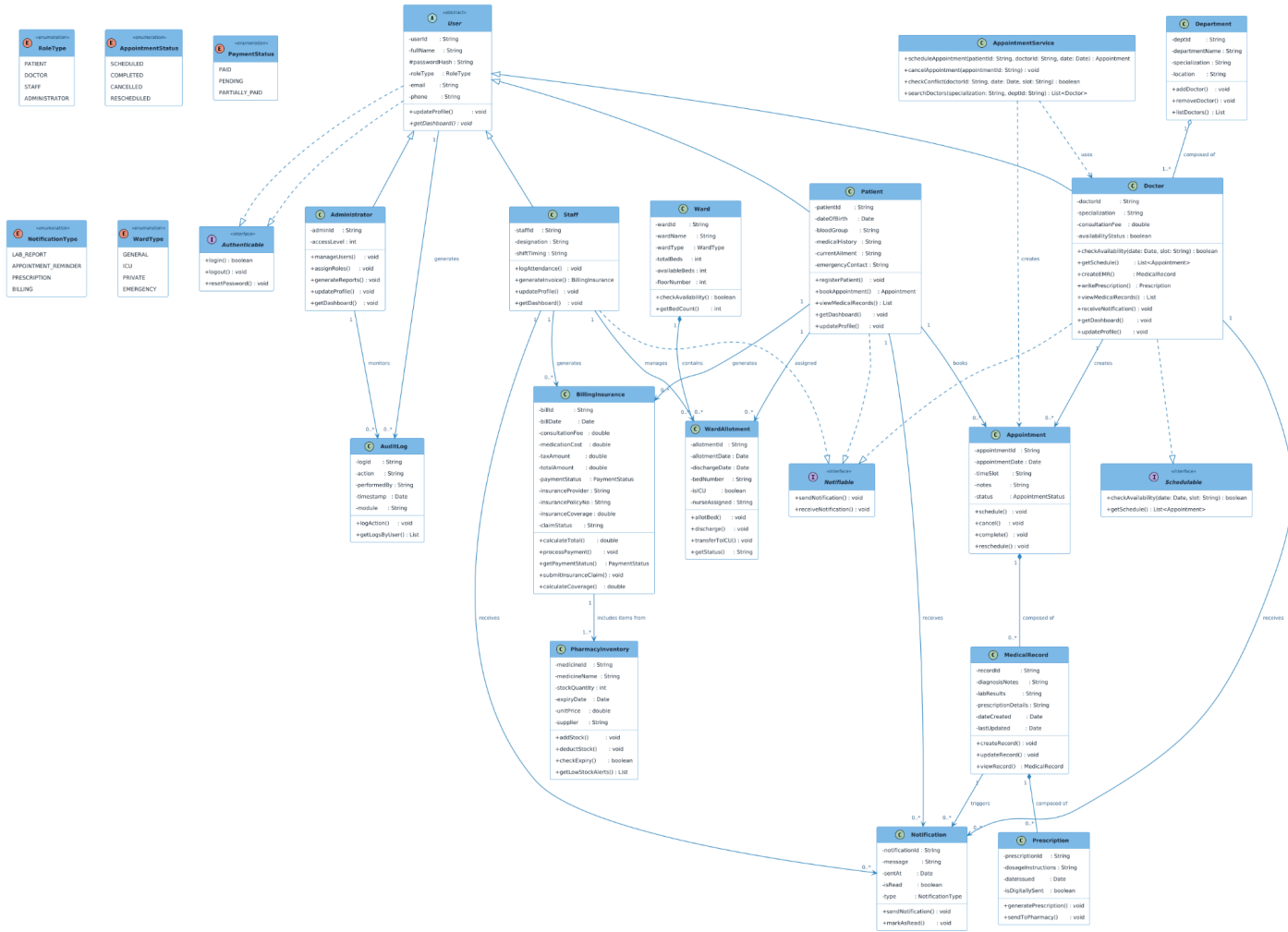
Class Diagram elements: BillingInsurance, Patient, Staff, Notification, BillingFacade

## 2.3 Additional Use Cases (Medium Priority)

| Use Case                                | Actor(s)             | Summary                                                                                                           |
|-----------------------------------------|----------------------|-------------------------------------------------------------------------------------------------------------------|
| UC5 — Manage Pharmacy Inventory (FR-07) | Staff                | Add medicines, update stock, flag low-stock items below reorder level. Auto-update after prescription dispensing. |
| UC6 — Lab Report Notification (FR-06)   | Staff/Nurse, Patient | Staff uploads lab results → System sends notification to patient (Observer fires on test completion).             |
| UC7 — Ward Allotment                    | Staff, Admin         | Allot a bed (Ward + bed number) to a patient. Track ACTIVE / DISCHARGED status.                                   |
| UC8 — Role-Based Authentication (FR-05) | All actors           | All modules include «include» User Authentication. RoleGuard validates X-User-Role header.                        |
| UC9 — Audit Logging (NFR-06)            | Administrator        | AuditLogService Singleton logs every action. Admin views 50 most recent log entries.                              |

Section 3 — Class Diagram

The Class Diagram shows the full HealthSync domain model. Below is a detailed description of every class, its attributes, methods, and relationships — exactly matching your diagram.



3.1 Enumerations (shown at top-left of diagram)

| Enumeration                     | Values                                                           | Used In                        |
|---------------------------------|------------------------------------------------------------------|--------------------------------|
| RoleType «enumeration»          | PATIENT, DOCTOR, STAFF, ADMINISTRATOR                            | User.roleType                  |
| AppointmentStatus «enumeration» | SCHEDULED, COMPLETED, CANCELLED, RESCHEDULED                     | Appointment.status             |
| PaymentStatus «enumeration»     | PAID, PENDING, PARTIALLY_PAID                                    | BillingInsurance.paymentStatus |
| NotificationType «enumeration»  | LAB_REPORT, APPOINTMENT_REMINDER, PRESCRIPTION, BILLING, GENERAL | Notification.type              |
| WardType «enumeration»          | GENERAL, ICU, PRIVATE, EMERGENCY                                 | Ward.wardType                  |



## 3.2 Class Descriptions

### 3.2.1 User (Abstract — «abstract»)

The central identity class. All actor classes (Patient, Doctor, Staff, Administrator) inherit from or relate to User. In the class diagram it is shown as an abstract class at the top with four extending concrete classes.

| Attribute / Method      | Type     | Description                                         |
|-------------------------|----------|-----------------------------------------------------|
| -userId                 | String   | Unique identifier, auto-generated (USR00001 format) |
| -fullName               | String   | Display name of the user                            |
| -passwordHash           | String   | SHA-256 hashed password                             |
| -roleType               | RoleType | Enum: PATIENT / DOCTOR / STAFF / ADMINISTRATOR      |
| -accessLevel            | int      | Numeric access level (higher = more access)         |
| -phone                  | String   | Contact phone number                                |
| +updateProfile() : void | Method   | Update name, phone, password                        |
| +getDashboard() : void  | Method   | Return role-specific dashboard data                 |

### 3.2.2 Patient (inherits from User)

| Attribute / Method                          | Type   | Description                                                                   |
|---------------------------------------------|--------|-------------------------------------------------------------------------------|
| -patientId                                  | String | Unique Patient ID (PAT00001 format), created by HealthSyncFactory             |
| -dateOfBirth                                | Date   | Patient's date of birth                                                       |
| -bloodGroup                                 | String | e.g. O+, AB-                                                                  |
| -medicalHistory                             | String | Free-text summary of past medical conditions                                  |
| -currentAilment                             | String | Present complaint / reason for visit                                          |
| -emergencyContact                           | String | Emergency contact phone number                                                |
| +registerPatient() : void                   | Method | Creates User + Patient via HealthSyncFactory; logs action via AuditLogService |
| +bookAppointment() : Appointment            | Method | Triggers AppointmentService.scheduleAppointment()                             |
| +viewMedicalRecords() : List<MedicalRecord> | Method | Fetches all MedicalRecords linked to this patient                             |
| +getDashboard() : void                      | Method | Returns patient dashboard: appointments, invoices, notifications              |
| +updateProfile() : void                     | Method | Updates patient profile details                                               |

### 3.2.3 Doctor (inherits from User)

| Attribute / Method | Type   | Description                        |
|--------------------|--------|------------------------------------|
| -doctorId          | String | Unique Doctor ID (DOC00001 format) |
| -specialization    | String | e.g. Cardiologist, Neurologist     |

| Attribute / Method                               | Type    | Description                                                     |
|--------------------------------------------------|---------|-----------------------------------------------------------------|
| -consultationFee                                 | double  | Per-consultation fee in INR                                     |
| -availabilityStatus                              | boolean | true = available for appointments                               |
| +checkAvailability(date, slot, String) : boolean | Method  | Check if a slot is free (isBooked = false)                      |
| +getSchedule() : List<Appointment>               | Method  | Returns all scheduled appointments for the doctor               |
| +createEMR() : MedicalRecord                     | Method  | Creates new MedicalRecord via HealthSyncFactory                 |
| +writePrescription() : Prescription              | Method  | Generates Prescription + PrescriptionItems                      |
| +viewMedicalRecords() : List                     | Method  | View all patient records authored by this doctor                |
| +getDashboard() : void                           | Method  | Returns doctor dashboard: today's appointments, pending records |
| +updateProfile() : void                          | Method  | Updates doctor profile                                          |

### 3.2.4 Staff (inherits from User)

| Attribute / Method                       | Type   | Description                                  |
|------------------------------------------|--------|----------------------------------------------|
| -staffId                                 | String | Unique Staff ID                              |
| -designation                             | String | e.g. Nurse, Lab Technician, Receptionist     |
| -shiftTiming                             | String | e.g. Morning / Evening / Night               |
| +logAttendance() : void                  | Method | Records staff attendance                     |
| +generateInvoice(...) : BillingInsurance | Method | Delegates to BillingFacade.generateInvoice() |
| +updateProfile() : void                  | Method | Updates staff profile                        |
| +getDashboard() : void                   | Method | Returns staff dashboard                      |

### 3.2.5 Administrator (inherits from User)

| Attribute / Method                       | Type   | Description                                     |
|------------------------------------------|--------|-------------------------------------------------|
| -adminId                                 | String | Unique Admin ID                                 |
| -accessLevel                             | int    | Full access (highest level)                     |
| +manageUsers() : void                    | Method | CRUD on User entities                           |
| +assignRoles() : void                    | Method | Set/update roleType for users                   |
| +generateReports() : void                | Method | View audit logs, system statistics              |
| +generateInvoice(...) : BillingInsurance | Method | Can also trigger billing                        |
| +updateProfile() : void                  | Method | Updates admin profile                           |
| +getDashboard() : void                   | Method | Admin dashboard: user counts, audit logs, stats |

### 3.2.6 Appointment

| Attribute / Method   | Type              | Description                                           |
|----------------------|-------------------|-------------------------------------------------------|
| -appointmentId       | String            | Unique Appointment ID (APT00001 format)               |
| -appointmentDate     | Date              | Date of the appointment                               |
| -timeSlot            | String            | Time slot string (e.g. 09:00–09:30)                   |
| -notes               | String            | Optional notes from patient or doctor                 |
| -status              | AppointmentStatus | Enum: SCHEDULED / COMPLETED / CANCELLED / RESCHEDULED |
| +schedule() : void   | Method            | Mark slot as booked, set status = SCHEDULED           |
| +cancel() : void     | Method            | Set status = CANCELLED, free the slot                 |
| +complete() : void   | Method            | Set status = COMPLETED                                |
| +reschedule() : void | Method            | Assign new slot, set status = RESCHEDULED             |

### 3.2.7 AppointmentService (Service class)

Coordinates UC2 — Appointment Scheduling. Uses Observer (notification) and Singleton (AuditLog). Associated with Doctor (1..\*) and Patient (1..\*).

| Method                                                              | Description                                                                                  |
|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| +scheduleAppointment(patientId, doctorId, date, slot) : Appointment | Validates availability, creates Appointment, fires Observer notification, logs via Singleton |
| +cancelAppointment(appointmentId) : void                            | Cancels appointment, frees slot, notifies patient                                            |
| +searchDoctors(specialization, deptId) : List<Doctor>               | Returns available doctors matching criteria                                                  |

### 3.2.8 MedicalRecord

| Attribute / Method            | Type   | Description                          |
|-------------------------------|--------|--------------------------------------|
| -recordId                     | String | Unique Record ID (REC00001 format)   |
| -diagnosisNotes               | String | Doctor's clinical diagnosis text     |
| -labResults                   | String | Lab test results summary             |
| -prescriptionDetails          | String | Summary of medicines prescribed      |
| -dateCreated                  | Date   | Date record was created              |
| -lastUpdated                  | Date   | Date record was last modified        |
| +createRecord() : void        | Method | Initialize record, set status = OPEN |
| +updateRecord() : void        | Method | Add notes, results; set lastUpdated  |
| +viewRecord() : MedicalRecord | Method | Fetch record for display             |

### 3.2.9 BillingInsurance

| Attribute / Method                  | Type          | Description                                      |
|-------------------------------------|---------------|--------------------------------------------------|
| -billId                             | String        | Unique Bill ID (BILL0001 format)                 |
| -billDate                           | Date          | Date invoice was generated                       |
| -consultationFee                    | double        | Doctor's consultation charge                     |
| -medicationCost                     | double        | Cost of medicines dispensed                      |
| -taxAmount                          | double        | Tax applied to total                             |
| -totalAmount                        | double        | Sum of all charges including tax                 |
| -paymentStatus                      | PaymentStatus | PAID / PENDING / PARTIALLY_PAID                  |
| -insuranceProvider                  | String        | Name of insurance company                        |
| -insurancePolicyNo                  | String        | Policy number                                    |
| -insuranceCoverage                  | double        | Amount covered by insurance                      |
| -claimStatus                        | String        | Status of insurance claim                        |
| +calculateTotal() : double          | Method        | Sum consultationFee + medicationCost + taxAmount |
| +processPayment() : void            | Method        | Mark as PAID, record payment method              |
| +getPaymentStatus() : PaymentStatus | Method        | Return current payment status                    |
| +submitInsuranceClaim() : void      | Method        | Submit claim to insurance provider               |
| +calculateCoverage() : double       | Method        | Compute net amount after insurance               |

### 3.2.10 PharmacyInventory

| Attribute / Method                  | Type   | Description                                  |
|-------------------------------------|--------|----------------------------------------------|
| -medicineId                         | String | Unique Medicine ID (MED00001 format)         |
| -medicineName                       | String | Brand/generic name of the medicine           |
| -stockQuantity                      | int    | Current units in stock                       |
| -expiryDate                         | Date   | Expiry date of the medicine batch            |
| -unitPrice                          | double | Price per unit in INR                        |
| -supplier                           | String | Name of the pharmaceutical supplier          |
| +addStock() : void                  | Method | Increase stockQuantity; log action           |
| +deductStock() : void               | Method | Decrease stockQuantity on dispensing         |
| +checkExpiry() : boolean            | Method | Return true if medicine is within expiry     |
| +getLowStockAlerts() : List<String> | Method | Return list of medicines below reorder level |

### 3.2.11 Ward

| Attribute / Method             | Type     | Description                         |
|--------------------------------|----------|-------------------------------------|
| -wardId                        | String   | Unique Ward ID                      |
| -wardName                      | String   | Ward name (e.g. ICU Block A)        |
| -wardType                      | WardType | GENERAL / ICU / PRIVATE / EMERGENCY |
| -totalBeds                     | int      | Total bed capacity                  |
| -availableBeds                 | int      | Currently unoccupied beds           |
| -floorNumber                   | int      | Floor where ward is located         |
| +checkAvailability() : boolean | Method   | Return availableBeds > 0            |
| +getBedCount() : int           | Method   | Return total and occupied bed count |

### 3.2.12 WardAllotment

| Attribute / Method      | Type    | Description                                   |
|-------------------------|---------|-----------------------------------------------|
| -allotmentId            | String  | Unique Allotment ID                           |
| -allotmentDate          | Date    | Date patient was admitted to the ward         |
| -dischargeDate          | Date    | Expected/actual discharge date                |
| -bedNumber              | String  | Specific bed assigned                         |
| -isICU                  | boolean | true if patient is in ICU                     |
| -nurseAssigned          | String  | Name/ID of assigned nurse                     |
| +allotBed() : void      | Method  | Assign bed; decrement availableBeds in Ward   |
| +discharge() : void     | Method  | Mark discharge; increment availableBeds       |
| +transferToICU() : void | Method  | Move patient to ICU ward; update isICU = true |
| +getStatus() : String   | Method  | Return ACTIVE or DISCHARGED                   |

### 3.2.13 Notification («interface» Notifiable)

Implements the Notifiable interface. Part of the Observer pattern — created by PersistentNotificationObserver when a subject fires an event.

| Attribute / Method         | Type             | Description                                                          |
|----------------------------|------------------|----------------------------------------------------------------------|
| -notificationId            | String           | Unique Notification ID                                               |
| -message                   | String           | Full notification body text                                          |
| -sentAt                    | Date             | Timestamp when notification was created                              |
| -isRead                    | boolean          | false until patient opens notification                               |
| -type                      | NotificationType | LAB_REPORT / APPOINTMENT_REMINDER / PRESCRIPTION / BILLING / GENERAL |
| +sendNotification() : void | Method           | Persist to notifications table (called by Observer)                  |
| +markAsRead() : void       | Method           | Set isRead = true                                                    |

### 3.2.14 AuditLog

| Attribute / Method      | Type   | Description                                          |
|-------------------------|--------|------------------------------------------------------|
| -logId                  | String | Unique Log ID                                        |
| -action                 | String | Action name: LOGIN, REGISTER, BOOK_APPOINTMENT, etc. |
| -performedBy            | String | UserId of the actor who triggered the action         |
| -timestamp              | Date   | Exact date-time of the action                        |
| -module                 | String | Module name: appointments, billing, pharmacy, etc.   |
| +logAction() : void     | Method | Write log entry to audit_log table (Singleton)       |
| +getLogsByUser() : List | Method | Retrieve all logs for a specific user                |

### 3.2.15 Department and Other Supporting Classes

| Class                      | Key Attributes                                                                  | Relationship                                          |
|----------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------|
| Department                 | deptId, departmentName, specialization, location                                | 1 Department contains 1..* Doctors                    |
| Prescription               | prescriptionId, dosageInstructions, dateIssued, isDigitallySent                 | Generated by MedicalRecord; sent to PharmacyInventory |
| Schedulable<br>«interface» | checkAvailability(date,slot,String):boolean;<br>getSchedule():List<Appointment> | Implemented by Appointment                            |

## 3.3 Key Class Relationships

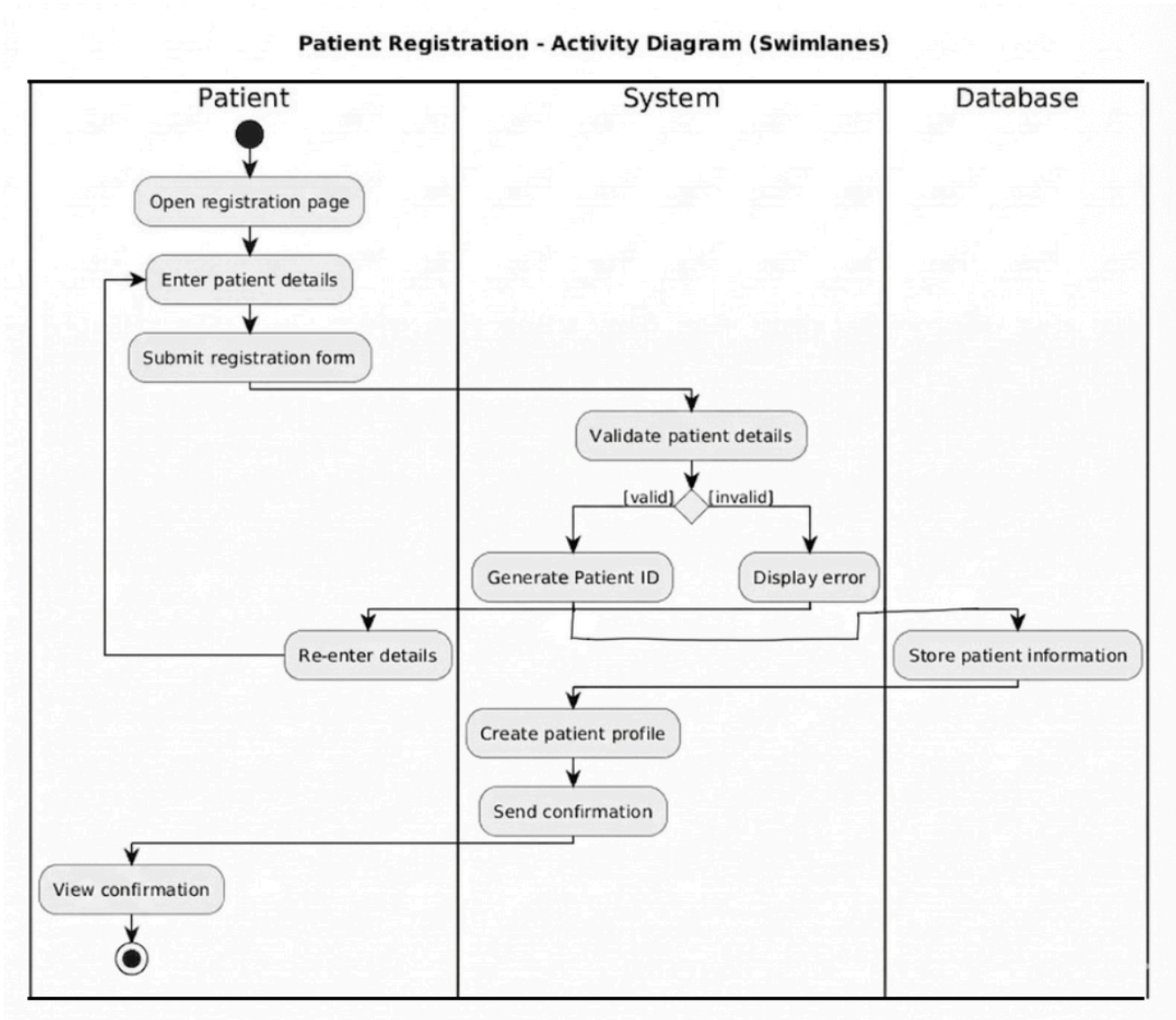
| Relationship                 | Type                                 | Multiplicity                                    |
|------------------------------|--------------------------------------|-------------------------------------------------|
| User → Patient               | Generalization (inheritance)         | User is the base; Patient extends User          |
| User → Doctor                | Generalization (inheritance)         | Doctor extends User with specialization         |
| User → Staff                 | Generalization (inheritance)         | Staff extends User with designation             |
| User → Administrator         | Generalization (inheritance)         | Administrator extends User with accessLevel     |
| Doctor → Department          | Association (many-to-one)            | Many Doctors belong to 1 Department             |
| Department → Doctor          | Aggregation                          | 1 Department contains 1..* Doctors              |
| Patient → Appointment        | Association                          | 0..* Appointments per Patient                   |
| Doctor → Appointment         | Association (via AppointmentService) | 1..* Appointments per Doctor                    |
| MedicalRecord → Prescription | Composition (composed of)            | 0..* Prescriptions per MedicalRecord            |
| MedicalRecord → Notification | Association (triggers)               | Completing a record triggers 0..* Notifications |

| Relationship                     | Type                              | Multiplicity                              |
|----------------------------------|-----------------------------------|-------------------------------------------|
| BillingInsurance → Notification  | Association (generates)           | Generating invoice fires notification     |
| Patient → BillingInsurance       | Association                       | Patient receives 0..* invoices            |
| Ward → WardAllotment             | Association (manages/contains)    | 1 Ward manages 0..* WardAllotments        |
| Patient → WardAllotment          | Association (assigned)            | Patient assigned to 0..* allotments       |
| Prescription → PharmacyInventory | Association (includes items from) | Prescription draws from PharmacyInventory |

## Section 4 — Activity Diagrams (4 Major Use Cases)

Activity diagrams describe step-by-step workflows for the four major use cases using swim lanes. Below is the complete textual description of each diagram plus its mapping to the implementation code.

### 4.1 Activity Diagram — UC1: Patient Registration (Swimlanes: Patient | System | Database)



**Code Mapping (AuthController.java):**

```
// POST /api/auth/register
// SYSTEM: Validate
if (userRepo.findByEmail(email).isPresent())
    return ResponseEntity.badRequest().body(Map.of("error", "Email already
registered"));

// SYSTEM: Generate Patient ID via Factory Method (Creational Pattern #1)
int nextUserNum = userRepo.findMaxUserId() + 1;
User user = HealthSyncFactory.createUser(nextUserNum, fullName, email, phone,
hash, RoleType.PATIENT);
userRepo.save(user); // DATABASE: Store user

int nextPatNum = patientRepo.findMaxPatientId() + 1;
Patient patient = HealthSyncFactory.createPatient(nextPatNum, user, bloodGroup,
emergency);
patientRepo.save(patient); // DATABASE: Store patient

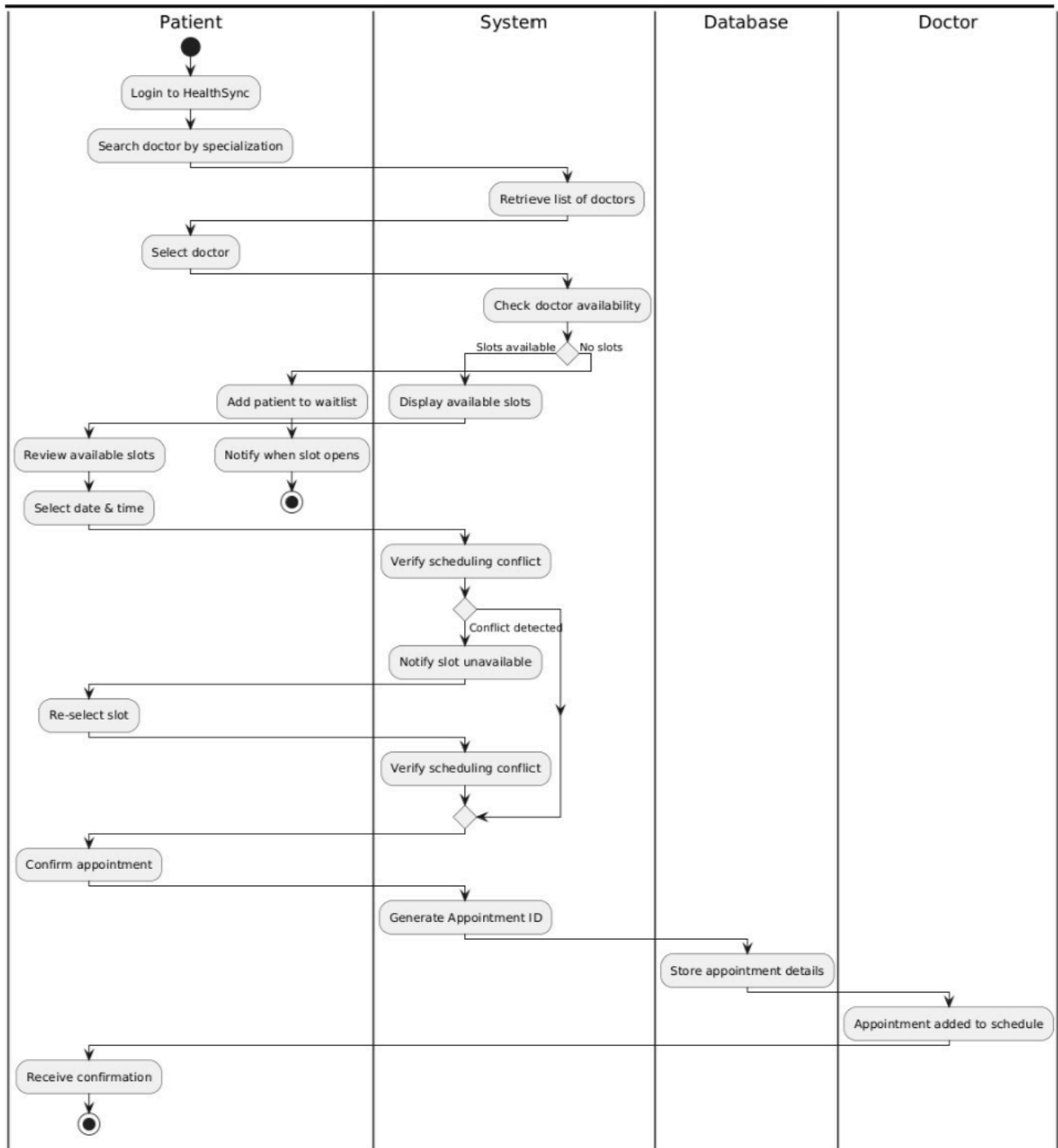
// SYSTEM: Audit log via Singleton (Creational Pattern #2)
auditLogService.logAction(userId, "REGISTER", "Patient registered: " +
fullName);

// SYSTEM: Send confirmation → Patient views confirmation
return ResponseEntity.ok(Map.of("patientId", patient.getPatientId(), "message",
"Registration successful"));
```

**4.2 Activity Diagram — UC2: Appointment Scheduling (Swimlanes: Patient | System | Database | Doctor)**



Appointment Scheduling - Activity Diagram (UC2)



### Code Mapping (AppointmentController.java + AppointmentService.java):

```

// GET /api/appointments/slots?doctorId=X&date=Y - SYSTEM: Check doctor
availability
slotRepo.findByDoctorDoctorIdAndSlotDateAndIsBookedFalse(doctorId, date);

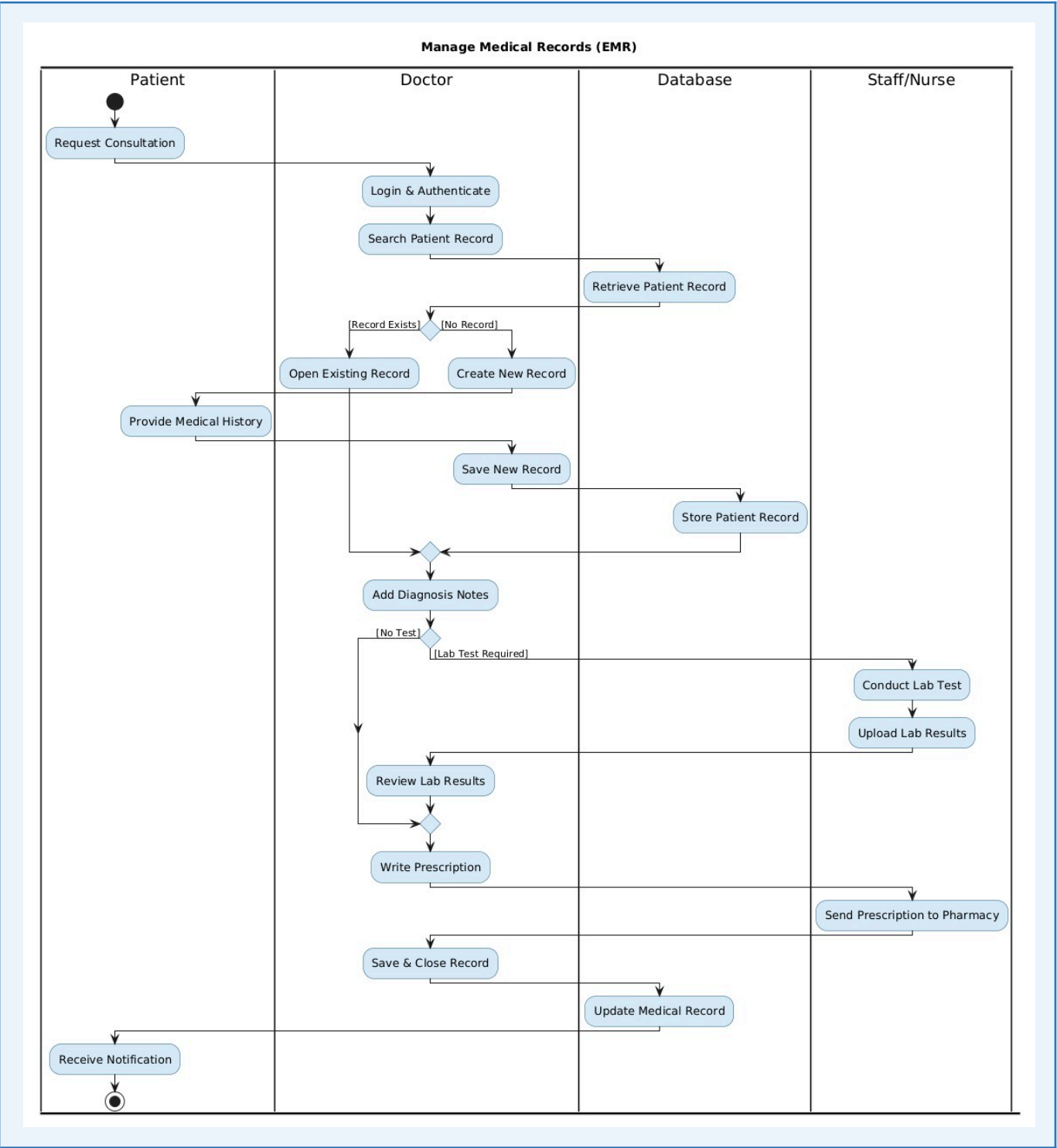
// POST /api/appointments - SYSTEM: Verify conflict + Generate Appointment ID
if (slot.getIsBooked()) // CONFLICT DETECTED
    return ResponseEntity.badRequest().body(Map.of("error", "Slot already
booked"));

// Generate Appointment ID
int nextId = apptRepo.findMaxAppointmentId() + 1;
appt.setAppointmentId(String.format("APT%05d", nextId));
    
```

```
appt.setStatus(AppointmentStatus.SCHEDULED);
apptRepo.save(appt); // DATABASE: Store appointment
slot.setIsBooked(true); slotRepo.save(slot); // Mark slot booked

// DOCTOR: Appointment added to schedule (slot.isBooked = true)
// Observer fires → Patient: Receive confirmation
notificationObserver.onEvent(patient.getUser(), APPOINTMENT, "Appointment
Confirmed", details);
```

4.3 Activity Diagram — UC3: Manage Medical Records / EMR (Swimlanes: Patient | Doctor | Database | Staff/Nurse)



**Code Mapping (MedicalRecordController.java + PrescriptionController.java + LabTestController.java):**

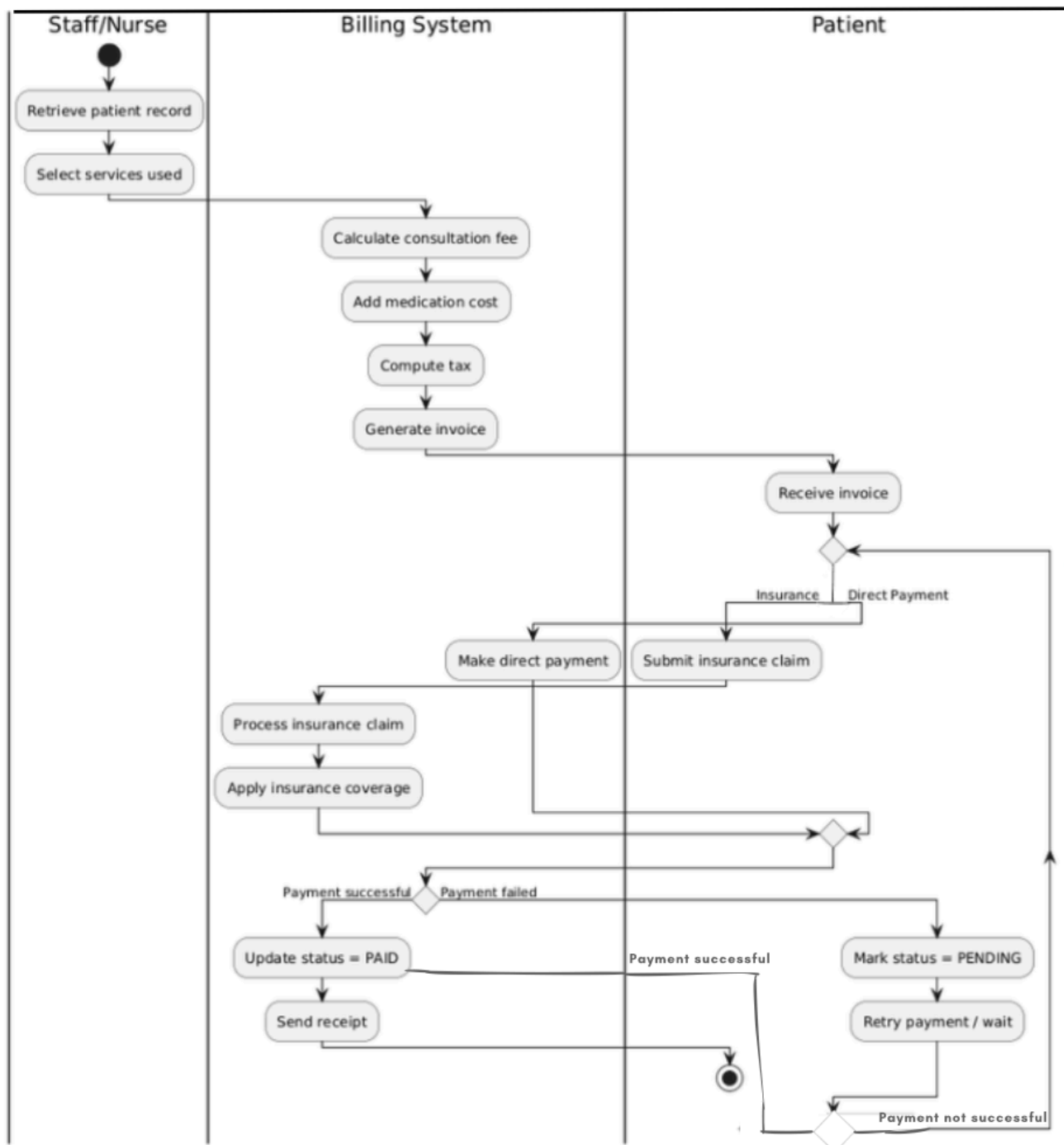
```
// POST /api/medical-records - DOCTOR: Create New Record
MedicalRecord rec = HealthSyncFactory.createMedicalRecord(nextId, patient, doctor,
fields);
rec.setStatus(RecordStatus.OPEN);
recordRepo.save(rec); // DATABASE: Store Patient Record

// POST /api/lab-tests - STAFF: Order Lab Test
labTest.setStatus(LabTestStatus.ORDERED);
labTestRepo.save(labTest);

// PUT /api/lab-tests/{id}/complete - STAFF: Upload Results
labTest.setStatus(LabTestStatus.COMPLETED);
labTest.setResultSummary(results);
labTest.setNotificationSent(true);
// Observer: PATIENT receives notification
notificationObserver.onEvent(patient.getUser(), LAB, "Lab Result Ready", testName);

// PUT /api/medical-records/{id}/close - DOCTOR: Save & Close Record
record.setStatus(RecordStatus.CLOSED);
recordRepo.save(record); // DATABASE: Update Medical Record
// PATIENT: Receive Notification (Observer)
```

**4.4 Activity Diagram — UC4: Billing & Insurance (Swimlanes: Staff/Nurse | Billing System | Patient)**



### Code Mapping (BillingFacade.java — Facade Pattern):

```

// BillingFacade.generateInvoice() - BILLING SYSTEM: Calculate + Generate
invoice
Invoice inv = HealthSyncFactory.createInvoice(nextId, patient, fields); //
Factory
inv.setPaymentStatus(PaymentStatus.PENDING);
invoiceRepo.save(inv);
// PATIENT: Receive invoice (Observer)
notificationObserver.onEvent(patient.getUser(), BILLING, "Invoice Generated",
total);
auditLogService.logAction("GENERATE_INVOICE", "invoices", inv.getBillId(),
desc); // Singleton

// BillingFacade.processPayment() - BILLING SYSTEM: Update status = PAID
inv.setPaymentStatus(PaymentStatus.PAID);

```

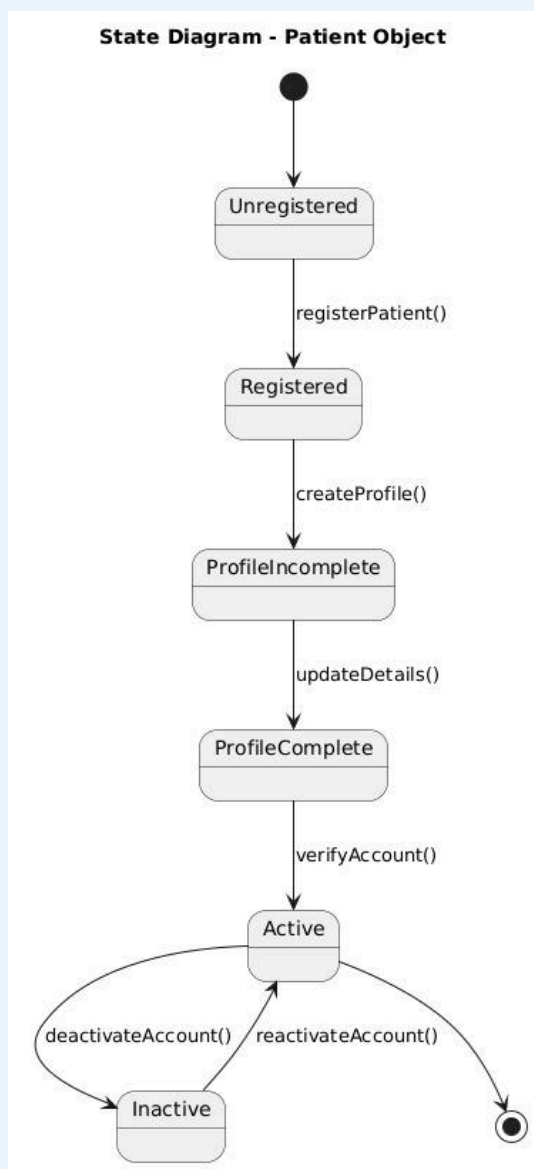
```
invoiceRepo.save(inv);  
// PATIENT: Receive receipt (Observer)  
notificationObserver.onEvent(patient.getUser(), BILLING, "Payment Received –  
Receipt", msg);  
// [Payment failed path] → status remains PENDING → Patient retries
```

## Section 5 — State Diagrams (4 Major Objects in Transition)

The four state diagrams describe the lifecycle of the four major objects that undergo state transitions in HealthSync.

### 5.1 State Diagram — Patient Object

This diagram tracks the lifecycle of a Patient from initial registration through account management.



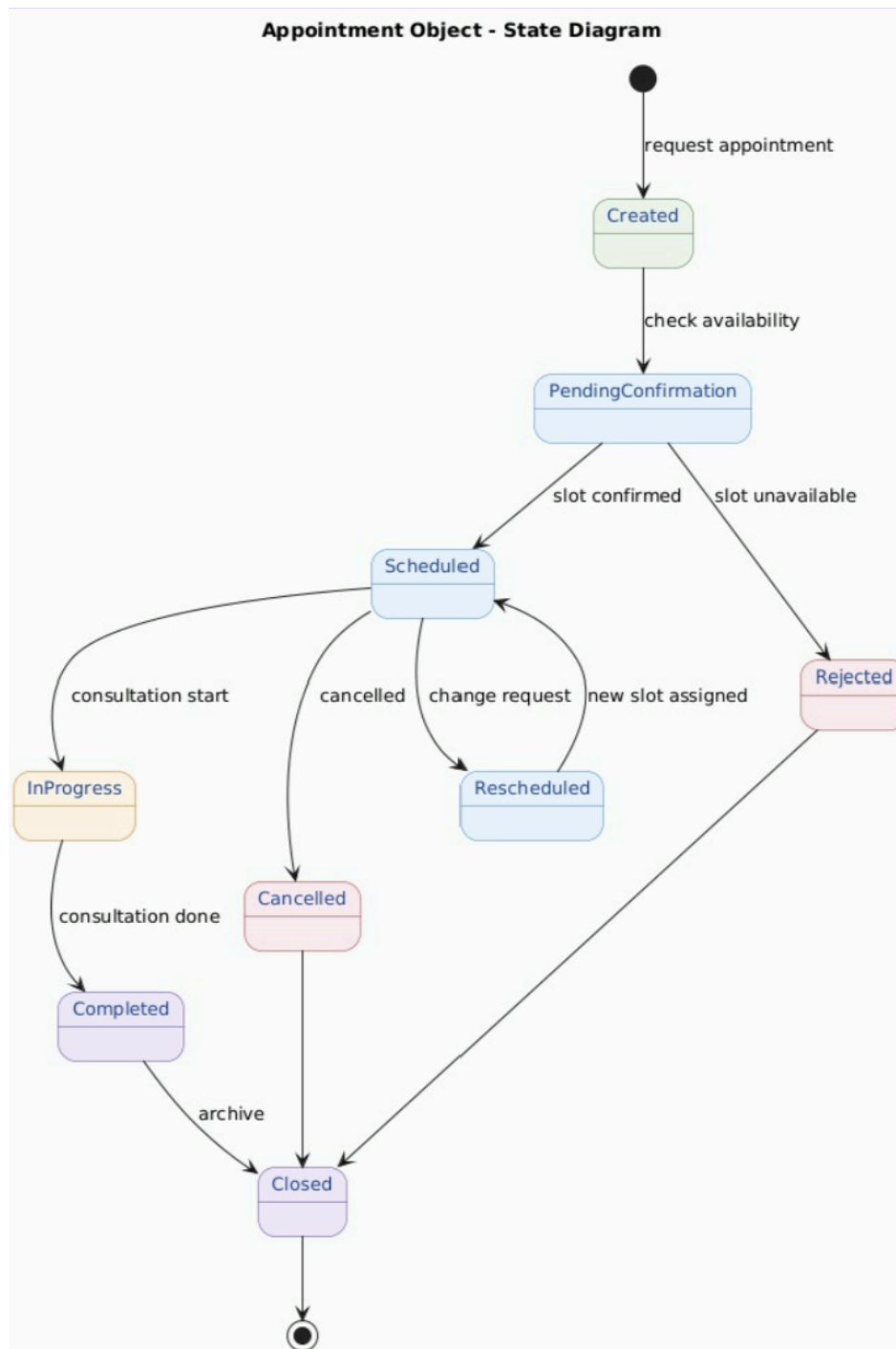
**Code Mapping:**

```
// User.java
private Boolean isActive = true; // Active state

// AuthController.java - registerPatient() transition
User user = HealthSyncFactory.createUser(...) // Unregistered → Registered
user.setIsActive(true); // → Active

// Admin can deactivate:
user.setIsActive(false); // Active → Inactive
user.setIsActive(true); // Inactive → Active (reactivate)
```

## 5.2 State Diagram — Appointment Object



This diagram tracks the appointment from request through archival. Directly maps to the AppointmentStatus enum in Appointment.java.

**Code Mapping:**

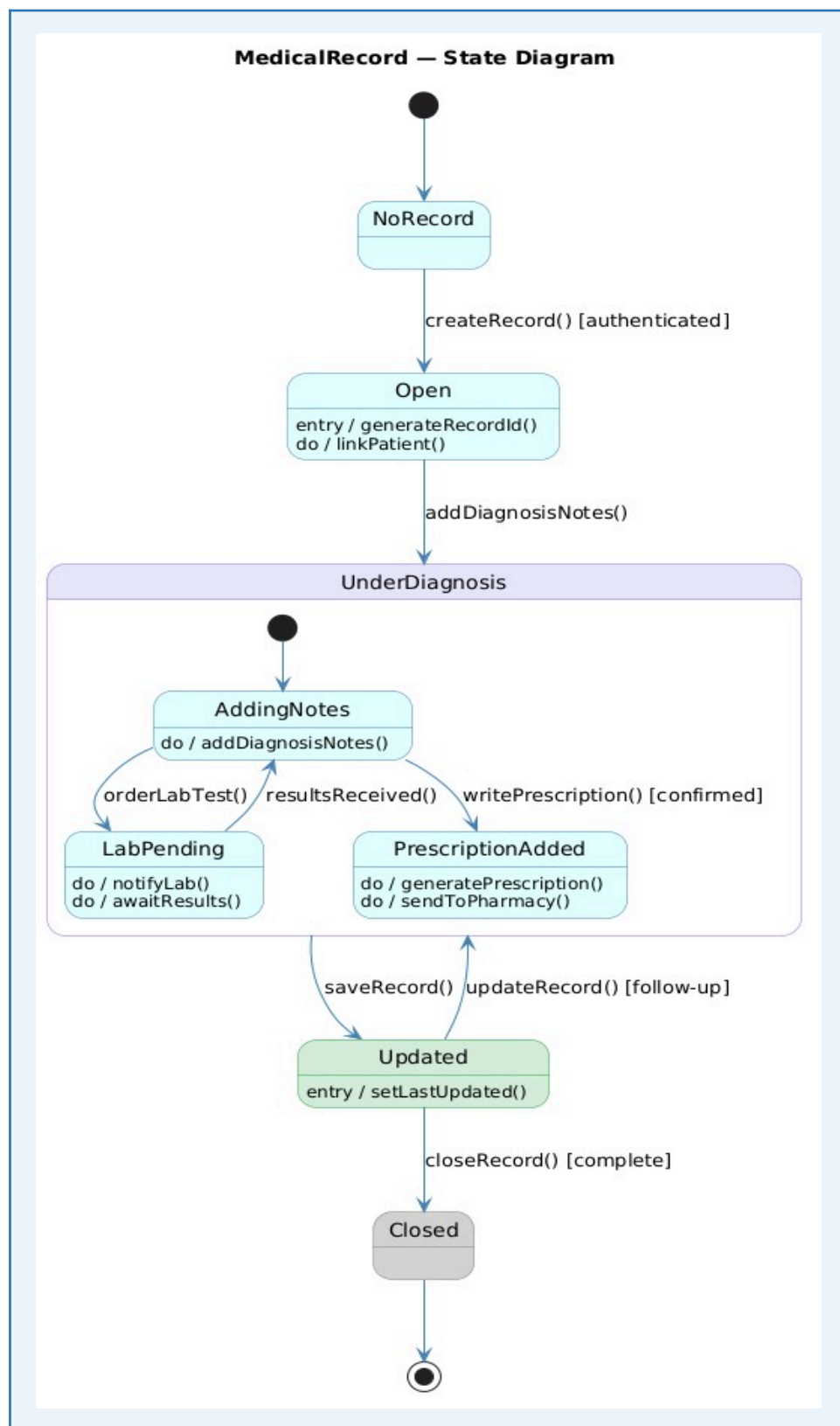
```
// Appointment.java
public enum AppointmentStatus {
    CREATED, PENDING_CONFIRMATION, SCHEDULED, IN_CONSULTATION,
    RESCHEDULED, COMPLETED, CANCELLED, WAITLISTED, CLOSED, FOLLOW_UP
}

// AppointmentController.java - state transitions via REST:
PUT /api/appointments/{id}/cancel    → status = CANCELLED
PUT /api/appointments/{id}/complete → status = COMPLETED
PUT /api/appointments/{id}/close     → status = CLOSED
PUT /api/appointments/{id}/follow-up → status = FOLLOW_UP

// Slot freed on cancel:
slot.setIsBooked(false); slotRepo.save(slot);
```

### 5.3 State Diagram — MedicalRecord Object

This is the advanced state diagram — it includes nested composite states for the UnderDiagnosis phase. Maps to MedicalRecord.status enum.

**Code Mapping:**

```

// MedicalRecord.java
public enum RecordStatus { OPEN, CLOSED }

// MedicalRecordController.java
// NoRecord → Open (createRecord [authenticated]):

```



```
MedicalRecord rec = HealthSyncFactory.createMedicalRecord(nextId, patient,
doctor, fields);
rec.setStatus(RecordStatus.OPEN); // Open state
recordRepo.save(rec);

// AddingNotes (addDiagnosisNotes):
rec.setDiagnosisNotes(notes); rec.setLastUpdated(LocalDate.now());

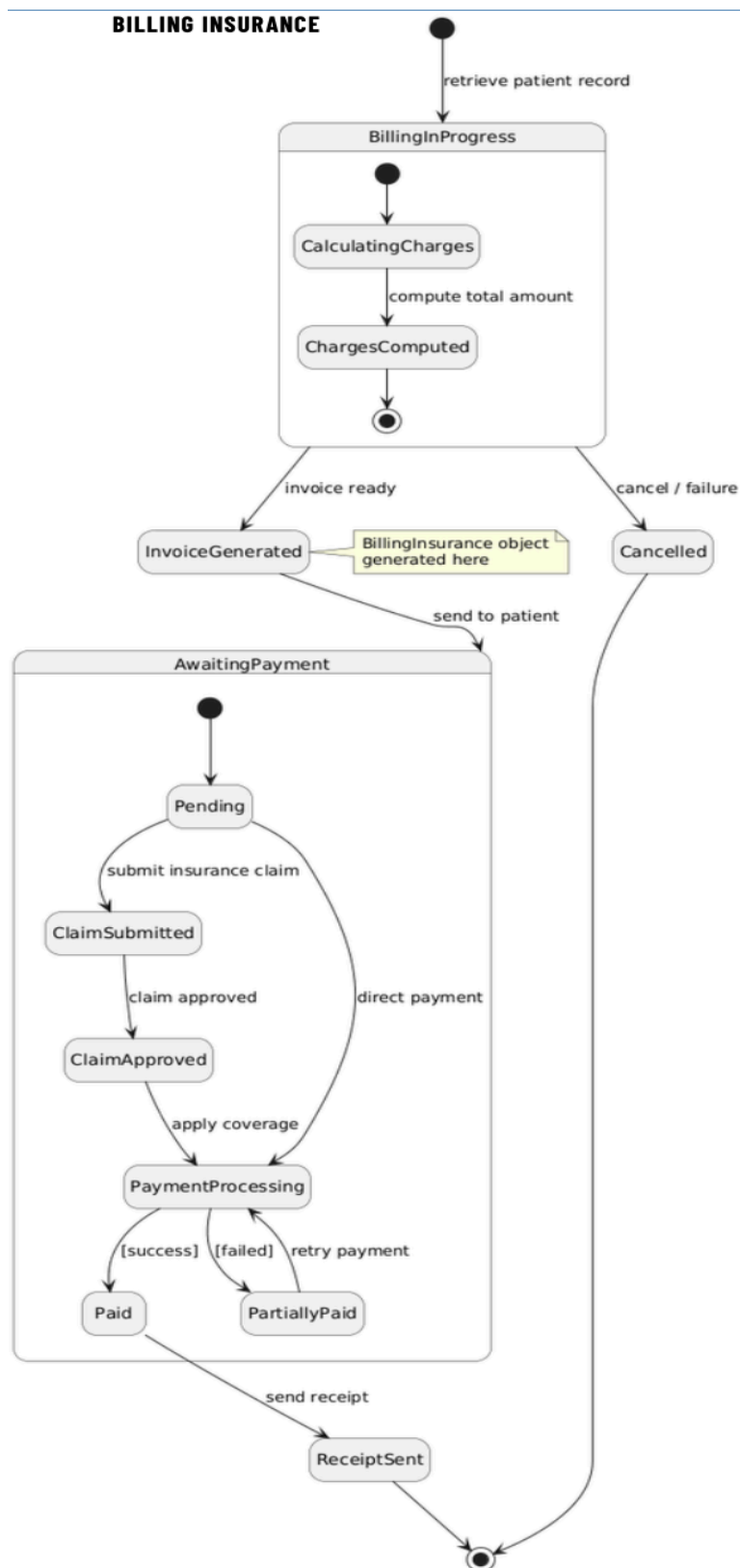
// LabPending (orderLabTest) → LabTest.status = ORDERED:
labTest.setStatus(LabTestStatus.ORDERED);

// PrescriptionAdded (writePrescription + sendToPharmacy):
prescription.setStatus(PrescriptionStatus.PENDING); // Sent to pharmacy

// Updated → Closed (closeRecord [complete]):
record.setStatus(RecordStatus.CLOSED);
recordRepo.save(record);
```

## 5.4 State Diagram — BillingInsurance Object

This diagram is the most detailed billing lifecycle, showing both insurance and direct payment paths with nested states inside the AwaitingPayment composite.



### Code Mapping:

```
// Invoice.java (maps to BillingInsurance in class diagram)
public enum PaymentStatus { PENDING, PAID, CANCELLED }

// BillingFacade.generateInvoice():
```

```

inv.setPaymentStatus(PaymentStatus.PENDING); // AwaitingPayment → Pending
invoiceRepo.save(inv); // InvoiceGenerated
notificationObserver.onEvent(..., "Invoice Generated", ...); // send to patient

// BillingFacade.processPayment():
inv.setPaymentStatus(PaymentStatus.PAID); // PaymentProcessing → Paid
invoiceRepo.save(inv);
notificationObserver.onEvent(..., "Payment Received – Receipt", ...); //
ReceiptSent

// PARTIALLY_PAID / insurance path tracked via:
// insuranceClaim, insuranceCoverage, claimStatus fields in Invoice

```

## Section 6 — MVC Architecture & Database Design

### 6.1 MVC Pattern in HealthSync

HealthSync implements the Model–View–Controller pattern across three distinct layers:

#### Model (com.healthsync.model + com.healthsync.repository)

The Model layer contains all domain entity classes annotated with `@Entity` (User, Patient, Doctor, Appointment, BillingInsurance/Invoice, MedicalRecord, Ward, WardAllotment, PharmacyInventory, Notification, AuditLog, Prescription, LabTest, Department, Staff). Each entity maps directly to a database table. Spring Data JPA repositories (`@Repository`) extend `JpaRepository<Entity, ID>` to provide CRUD plus custom query methods like `findByPatientPatientId()`, `findMaxBillId()`, etc.

```

// Example: Patient.java – Model class
@Entity @Table(name = "patients")
public class Patient {
    @Id @Column(name = "patient_id") private String patientId; // PAT00001
    @OneToOne @JoinColumn(name = "user_id") private User user;
    private String bloodGroup, medicalHistory, emergencyContact;
    private LocalDate dateOfBirth;
}

// Example: PatientRepository.java – Repository (Model support)
@Repository
public interface PatientRepository extends JpaRepository<Patient, String> {
    Optional<Patient> findByUserEmail(String email);
    @Query("SELECT COALESCE(MAX(CAST(SUBSTRING(p.patientId, 4) AS int)), 0) FROM Patient p")
    int findMaxPatientId();
}

```

#### View (src/main/resources/static/index.html)

The View is a Single-Page Application (SPA). One `index.html` file contains the entire UI — login, register, and all dashboard panels (patient, doctor, staff, admin). JavaScript `fetch()` calls invoke the REST API endpoints and update the DOM dynamically with JSON responses. There is no server-side HTML rendering; the view purely consumes the JSON returned by Controllers.

```

// index.html – View layer (JavaScript fetch call example)
async function loadAppointments() {
    const res = await fetch('/api/appointments/patient/' +
        currentUser.patientId,
        { headers: { 'X-User-Role': currentUser.role } });
}

```

```
const data = await res.json();
renderAppointmentsTable(data); // Update DOM
}
```

Controller (com.healthsync.controller)

Each domain area has a dedicated @RestController that receives HTTP requests, applies role-based access via RoleGuard, delegates to repositories/services/facades, and returns JSON responses. Controllers never contain business logic — they are the HTTP entry point only.

| Controller              | Base Endpoint        | Key Operations                                                       |
|-------------------------|----------------------|----------------------------------------------------------------------|
| AuthController          | /api/auth            | POST /login, POST /register, password validation                     |
| AppointmentController   | /api/appointments    | GET all/by-patient/by-doctor, POST create, PUT cancel/complete/close |
| MedicalRecordController | /api/medical-records | GET, POST create, PUT update/close                                   |
| BillingController       | /api/billing         | POST generate-invoice (→ BillingFacade), POST process-payment        |
| PharmacyController      | /api/pharmacy        | GET inventory, POST add-medicine, PUT update-stock                   |
| LabTestController       | /api/lab-tests       | POST order, PUT start/complete/cancel, GET results                   |
| PrescriptionController  | /api/prescriptions   | POST create, PUT dispense/cancel                                     |
| WardController          | /api/wards           | GET wards, POST add-ward, POST allot-bed, PUT discharge              |
| DoctorController        | /api/doctors         | GET all doctors, GET by specialization                               |
| PatientController       | /api/patients        | GET all patients, GET by ID                                          |
| DashboardController     | /api/dashboard       | GET stats: patient count, appointment count, bed occupancy           |

6.2 MVC Request Flow

**End-to-End Request Flow (e.g., Book Appointment)**

- VIEW (index.html JS): fetch('POST /api/appointments', body={patientId, doctorId, slotId, date}) + header X-User-Role: PATIENT
- CONTROLLER (AppointmentController.create()):
  - RoleGuard.requireRole(role, "PATIENT", "STAFF") — access check
  - Fetch Patient, Doctor, Slot from repositories (MODEL)
  - Validate slot not booked
  - Create Appointment entity, save to DB
- SERVICE / PATTERN LAYER:
  - NotificationObserver.onEvent() — Observer fires, saves Notification (MODEL)
  - AuditLogService.logAction() — Singleton logs action (MODEL)
- CONTROLLER returns: ResponseEntity.ok(Map<String,Object>) → JSON

5. VIEW (JS): Receives JSON → Updates appointment table in DOM

## 6.3 Database Design — Table Summary

| Table                       | Primary Key               | Key Foreign Keys                       | Maps To Class              |
|-----------------------------|---------------------------|----------------------------------------|----------------------------|
| users                       | user_id (USR00001)        | —                                      | User                       |
| patients                    | patient_id (PAT00001)     | user_id → users                        | Patient                    |
| doctors                     | doctor_id (DOC00001)      | user_id → users, dept_id → departments | Doctor                     |
| departments                 | dept_id (DEPT001)         | hod_user_id → users                    | Department                 |
| staff                       | staff_id                  | user_id → users, dept_id → departments | Staff                      |
| doctor_availability_slots   | slot_id (auto)            | doctor_id → doctors                    | DoctorAvailability Slot    |
| appointments                | appointment_id (APT00001) | patient_id, doctor_id, slot_id         | Appointment                |
| medical_records             | record_id (REC00001)      | patient_id, doctor_id                  | MedicalRecord              |
| lab_tests                   | test_id (auto)            | record_id, ordered_by → doctors        | LabTest                    |
| prescriptions               | prescription_id           | record_id, doctor_id, patient_id       | Prescription               |
| prescription_items          | item_id (auto)            | prescription_id, medicine_id           | PrescriptionItem           |
| pharmacy_inventory          | medicine_id (MED00001)    | —                                      | PharmacyInventory          |
| invoices (BillingInsurance) | bill_id (BILL0001)        | patient_id, appointment_id             | BillingInsurance / Invoice |
| payments                    | payment_id (auto)         | invoice_id → invoices                  | Payment                    |
| wards                       | ward_id                   | —                                      | Ward                       |
| ward_allotments             | allotment_id (auto)       | patient_id, ward_id                    | WardAllotment              |
| notifications               | notif_id (auto)           | user_id → users                        | Notification               |
| audit_log                   | log_id (auto)             | —                                      | AuditLog                   |

## Section 7 — SOLID & GRASP Principles

### 7.1 SOLID Principles

#### S — Single Responsibility Principle

| Class                | Single Responsibility                                         | Evidence                                                                              |
|----------------------|---------------------------------------------------------------|---------------------------------------------------------------------------------------|
| AuditLogService      | Only writes and retrieves audit log entries                   | Has only logAction() and getRecentLogs() — no billing, scheduling, or UI logic        |
| BillingFacade        | Only orchestrates the billing workflow                        | Delegates HTTP parsing to BillingController; does not handle slots or medical records |
| HealthSyncFactory    | Only constructs domain entity objects with auto-generated IDs | Has only static factory methods — no repository calls other than sequence numbers     |
| RoleGuard            | Only validates role-based access                              | One method: requireRole(role, ...allowedRoles) — returns null or 403 ResponseEntity   |
| Each @RestController | Only handles HTTP layer — parse request, return response      | Controllers call repos/services; they contain no business rules themselves            |

#### O — Open/Closed Principle

| Example                        | Open For Extension                                                                    | Closed For Modification                                            |
|--------------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| HealthSyncFactory              | Add new factory methods (e.g., createLabTest()) for new entities                      | Existing createPatient(), createInvoice() etc. are never modified  |
| NotificationObserver interface | New channels: EmailObserver, SMSObserver can be added by implementing the interface   | Existing subjects (AppointmentService, BillingFacade) not modified |
| AppointmentStatus enum         | New states (WAITLISTED, FOLLOW_UP) added to enum without changing state-handling code | Existing statuses and their handlers remain unchanged              |

#### L — Liskov Substitution Principle

- PersistentNotificationObserver implements NotificationObserver. Any code @Autowired NotificationObserver (BillingFacade, AppointmentService) can work with any concrete implementation — they are substitutable.
- All JPA repositories extend JpaRepository<Entity, ID> — substitutable at the interface level.
- Patient, Doctor, Staff, Administrator all inherit from User — they can be substituted wherever a User reference is expected (e.g., Notification.user).

#### I — Interface Segregation Principle

- NotificationObserver defines exactly one method: onEvent(User, type, title, message). Implementors are not forced to implement irrelevant methods.
- Schedulable interface in the class diagram defines only checkAvailability() and getSchedule() — not a bloated interface.

- Each Repository interface only declares the query methods needed for its domain entity.

## D — Dependency Inversion Principle

- BillingFacade and AppointmentService depend on the NotificationObserver interface (abstraction), not on PersistentNotificationObserver (concretion). Spring injects the concrete class via `@Autowired`.
- All `@RestController`s depend on JpaRepository interfaces, not on Hibernate or H2 implementations.
- HealthSyncFactory is a static utility — it depends on domain model classes, not on their persistence mechanisms.

## 7.2 GRASP Principles

| Principle          | Application in HealthSync                                                                                                                                                                     | Where in Code                                                                                                      |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Information Expert | Each class manages its own data. Invoice knows how to calculateTotal(). AuditLog knows its own logTime. PharmacyInventory knows its own stockQuantity.                                        | Invoice.calculateTotal(),<br>AuditLog.getLogByUser(),<br>PharmacyInventory.getLowStockAlerts()                     |
| Creator (GRASP)    | HealthSyncFactory is the designated Creator — it has all initialisation data needed for Patient, User, Invoice, MedicalRecord, PharmacyInventory. No controller does 'new Entity()' directly. | HealthSyncFactory.createPatient(),<br>createUser(), createInvoice(),<br>createMedicalRecord(),<br>createMedicine() |
| Controller (GRASP) | RoleGuard is the system-level GRASP Controller intercepting every HTTP request. AppointmentService is the use-case controller for UC2.                                                        | RoleGuard.requireRole(),<br>AppointmentService.scheduleAppointment()                                               |
| Low Coupling       | BillingController is decoupled from 4 repositories and Observer — it only calls BillingFacade. AppointmentController does not know about notifications or audit logging.                      | BillingController has one <code>@Autowired</code> BillingFacade; AppointmentController has no notification code    |
| High Cohesion      | BillingFacade only contains billing-related methods. AuditLogService only contains audit methods. Each controller handles only its domain.                                                    | All methods in BillingFacade relate to invoice/payment lifecycle                                                   |
| Polymorphism       | NotificationObserver uses polymorphism — different concrete observers handle delivery differently. AppointmentStatus enum enables polymorphic status checks.                                  | PersistentNotificationObserver implements NotificationObserver;<br>AppointmentStatus.CANCELLED, .COMPLETED etc.    |
| Indirection        | BillingFacade introduces indirection between BillingController and the 4-repository billing subsystem.                                                                                        | BillingController only knows BillingFacade; it has no reference to InvoiceRepository or PaymentRepository          |
| Pure Fabrication   | HealthSyncFactory and AuditLogService don't represent real-world entities — they are invented                                                                                                 | com.healthsync.factory.HealthSyncFactory;<br>com.healthsync.singleton.AuditLogService                              |

| Principle | Application in HealthSync                             | Where in Code |
|-----------|-------------------------------------------------------|---------------|
|           | service classes to achieve cohesion and low coupling. |               |

## Section 8 — Design Patterns Implementation

HealthSync implements 4 design patterns: 2 Creational (Factory Method + Singleton), 1 Structural (Facade), 1 Behavioral (Observer). Each pattern is described with intent, code implementation, and UI screenshot guide.

### 8.1 Creational Pattern #1 — Factory Method (HealthSyncFactory.java)

#### Intent

Define an interface for creating objects, but let the factory decide which class to instantiate and how to construct it. Remove object creation logic from controllers and centralise it in one place.

#### Problem It Solves

Before the factory, controllers directly wrote 'new Patient()' + inline String.format() for ID generation — this logic was duplicated across AuthController, BillingController, PharmacyController, and MedicalRecordController. Any change to ID format (e.g., from PAT%05d to PAT-%05d) required modifying 4 controllers. This violates SRP and OCP.

#### Implementation in Code

```
// com.healthsync.factory.HealthSyncFactory
public class HealthSyncFactory {
    private HealthSyncFactory() {} // No instantiation — static utility

    // Factory method 1: Create User
    public static User createUser(int nextNum, String fullName, String email,
                                String phone, String passwordHash,
                                User.RoleType role) {
        User u = new User();
        u.setUserId(String.format("USR%05d", nextNum)); // ID generation
        centralised here
        u.setFullName(fullName); u.setEmail(email); u.setPhone(phone);
        u.setPasswordHash(passwordHash); u.setRoleType(role);
        u.setIsActive(true);
        return u; // Caller just gets a ready-to-persist object
    }

    // Factory method 2: Create Patient
    public static Patient createPatient(int nextNum, User user,
                                       String bloodGroup, String emergency) {
        Patient p = new Patient();
        p.setPatientId(String.format("PAT%05d", nextNum));
        p.setUser(user); p.setBloodGroup(bloodGroup != null ? bloodGroup : "");
        p.setEmergencyContact(emergency != null ? emergency : "");
        return p;
    }

    // Factory method 3: Create Invoice (BillingInsurance)
    public static Invoice createInvoice(int nextNum, Patient patient,
                                       Map<String,String> fields) {
        Invoice inv = new Invoice();
        inv.setBillId(String.format("BILL%04d", nextNum));
    }
}
```



```

        inv.setPatient(patient);
        inv.setConsultationFee(decimal(fields, "consultationFee"));
        inv.setMedicationCost(decimal(fields, "medicationCost"));
        inv.setTaxAmount(decimal(fields, "taxAmount"));
        inv.setPaymentStatus(Invoice.PaymentStatus.PENDING);
        return inv;
    }

    // Also: createMedicalRecord(), createMedicine() - all centralised here
}

```

### Usage in Controllers (GRASP Creator in action):

```

// AuthController.java - controller calls factory, not 'new User()'
User user = HealthSyncFactory.createUser(nextUserNum, fullName, email, phone,
hash, RoleType.PATIENT);
userRepo.save(user);
Patient patient = HealthSyncFactory.createPatient(nextPatNum, user, bloodGroup,
emergency);
patientRepo.save(patient);

// BillingFacade.java - invoice creation via factory
Invoice inv = HealthSyncFactory.createInvoice(invoiceRepo.findMaxBillId() + 1,
patient, fields);
invoiceRepo.save(inv);

```

## 8.2 Creational Pattern #2 — Singleton (AuditLogService.java)

### Intent

Ensure that a class has only ONE instance throughout the application lifetime, and provide a global point of access to it.

### Why Singleton Here

The AuditLog is the single source of truth for all system events. Multiple instances would lead to non-sequential audit entries, lost log records, or different controllers writing to different buffers. Spring's `@Component` with default scope automatically ensures exactly one bean instance is created and injected everywhere — the Spring IoC container is the Singleton registry.

### Implementation in Code

```

// com.healthsync.singleton.AuditLogService
@Component // ← Spring makes this a Singleton bean automatically
public class AuditLogService {

    @Autowired private AuditLogRepository auditLogRepo;

    // Proof of single instantiation
    private static int instanceCount = 0;

    @PostConstruct
    public void init() {
        instanceCount++;
        System.out.println("[AuditLogService] Singleton instance #" +
instanceCount + " created.");
        // If this ever prints > 1, the Singleton guarantee is broken.
    }

    public void logAction(String userId, String action,

```

```

        String tableName, String recordRef, String
description) {
    AuditLog log = new AuditLog();
    log.setUserId(userId); log.setAction(action);
    log.setTableName(tableName); log.setRecordRef(recordRef);
    log.setDescription(description); log.setLogTime(LocalDateTime.now());
    auditLogRepo.save(log); // Writes to audit_log table
}

// Same @Autowired instance injected in:
// AuthController, BillingFacade, AppointmentService, MedicalRecordController,
// etc.

```

## 8.3 Structural Pattern — Facade (BillingFacade.java)

### Intent

Provide a simplified, unified interface to a complex subsystem. The BillingFacade hides the complexity of coordinating 4 repositories + Observer + Singleton behind two simple methods: generateInvoice() and processPayment().

### Why Facade Here

The billing workflow involves: PatientRepository (fetch patient), InvoiceRepository (save invoice), PaymentRepository (save payment), NotificationObserver (notify patient), and AuditLogService (log action). Without the Facade, BillingController would need to @Autowire all 5 dependencies and contain all orchestration logic — violating SRP. With the Facade, BillingController has exactly ONE dependency: BillingFacade.

### Implementation in Code

```

// com.healthsync.facade.BillingFacade - Structural Pattern
@Component
public class BillingFacade {
    // 5 dependencies hidden from controller:
    @Autowired private InvoiceRepository invoiceRepo;
    @Autowired private PaymentRepository paymentRepo;
    @Autowired private PatientRepository patientRepo;
    @Autowired private AuditLogService auditLogService; // Singleton
    @Autowired private NotificationObserver notificationObserver; // Observer

    // Simple method 1: generateInvoice (State: BillingInProgress →
    // InvoiceGenerated → AwaitingPayment)
    public Invoice generateInvoice(Map<String, String> fields) {
        Patient patient =
        patientRepo.findById(fields.get("patientId")).orElseThrow();
        Invoice inv =
        HealthSyncFactory.createInvoice(invoiceRepo.findMaxBillId()+1, patient, fields);
        invoiceRepo.save(inv); // BillingInsurance
        created
        notificationObserver.onEvent(patient.getUser(), // Observer: patient
        notified
        Notification.NotifType.BILLING, "Invoice Generated", "Total: " +
        inv.getTotalAmount());
        auditLogService.logAction("GENERATE_INVOICE", // Singleton: logged
        "invoices", inv.getBillId(), "Invoice for " +
        patient.getUser().getFullName());
        return inv;
    }
}

```

```
// Simple method 2: processPayment (State: AwaitingPayment → Paid →
ReceiptSent)
public Invoice processPayment(String billId, String amount, String method,
String refNo) {
    Invoice inv = invoiceRepo.findById(billId).orElseThrow();
    Payment payment = new Payment();
    payment.setInvoice(inv); payment.setAmountPaid(new BigDecimal(amount));
    payment.setPaymentMethod(Payment.PaymentMethod.valueOf(method));
    paymentRepo.save(payment);
    inv.setPaymentStatus(Invoice.PaymentStatus.PAID); // State → PAID
    invoiceRepo.save(inv);
    notificationObserver.onEvent(inv.getPatient().getUser(), // Observer:
receipt sent
        Notification.NotifType.BILLING, "Payment Received - Receipt",
        "Payment of Rs." + amount + " via " + method);
    auditLogService.logAction("PROCESS_PAYMENT", "payments", billId,
"Payment " + amount); // Singleton
    return inv;
}

// BillingController.java - only sees the Facade:
@Autowired private BillingFacade billingFacade; // ONE dependency only
Invoice inv = billingFacade.generateInvoice(body); // One call - all complexity
hidden
```

## 8.4 Behavioral Pattern — Observer (NotificationObserver)

### Intent

Define a one-to-many dependency so that when a subject (Appointment, Invoice, LabTest) changes state, all registered observers are notified and updated automatically. Decouples the subject from how notifications are delivered.

### Why Observer Here

Multiple subjects (AppointmentService, BillingFacade, LabTestController) need to notify patients when state changes occur. Without Observer, each subject would embed notification code inside business logic — violating SRP and OCP. Adding an Email channel would require modifying every subject class. With Observer, subjects just call `onEvent()` on the interface — the concrete observer handles delivery.

### Implementation in Code

```
// com.healthsync.observer.NotificationObserver - OBSERVER INTERFACE
public interface NotificationObserver {
    void onEvent(User user, Notification.NotifType type, String title, String
message);
}

// com.healthsync.observer.PersistentNotificationObserver - CONCRETE OBSERVER
@Component
public class PersistentNotificationObserver implements NotificationObserver {
    @Autowired private NotificationRepository notifRepo;

    @Override
    public void onEvent(User user, Notification.NotifType type, String title,
String message) {
        Notification n = new Notification();
        n.setUser(user); n.setType(type);
        n.setTitle(title); n.setMessage(message);
```

```
n.setIsRead(false);
notifRepo.save(n); // Persists to notifications table
}

// SUBJECTS - call the observer on state change:

// AppointmentService.java (UC2: Scheduled state)
notificationObserver.onEvent(patient.getUser(), APPOINTMENT,
    "Appointment Confirmed",
    "Your appointment with Dr." + doctor.getUser().getFullName() + " on " + date
+ " is confirmed.");

// BillingFacade.java (UC4: InvoiceGenerated state)
notificationObserver.onEvent(patient.getUser(), BILLING,
    "Invoice Generated", "Invoice " + billId + " for Rs." + total + " is
ready.");

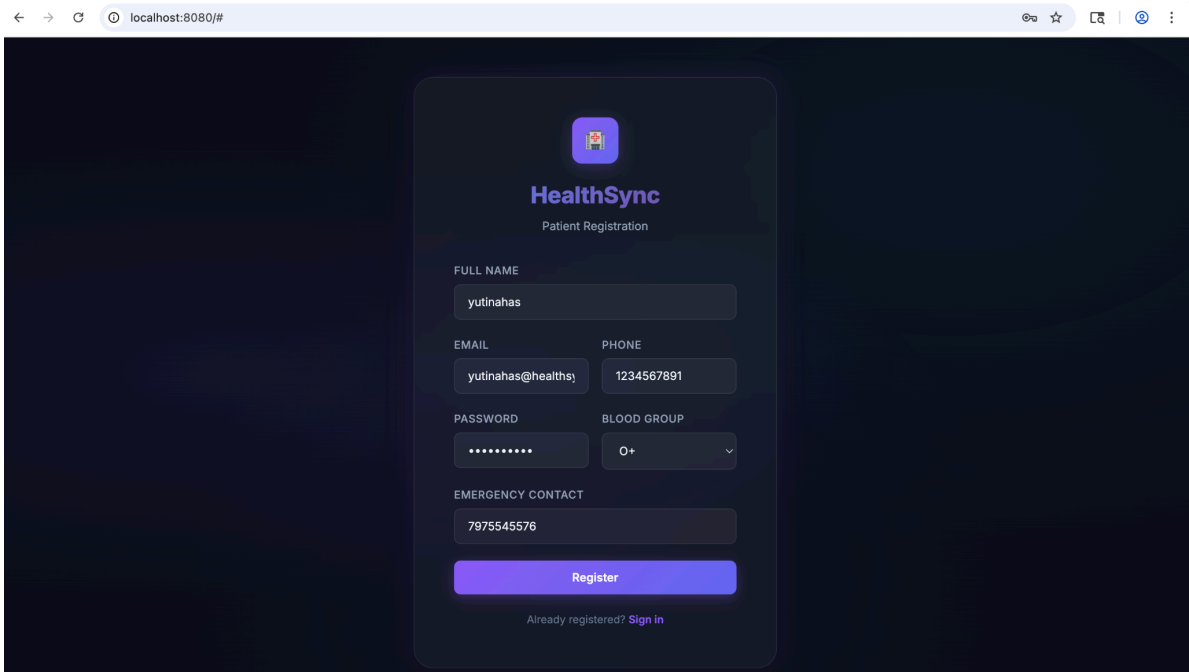
// LabTestController.java (UC6: COMPLETED state)
notificationObserver.onEvent(patient.getUser(), LAB,
    "Lab Result Ready", testName + ": " + resultSummary);
```

Observer triggers mapped to State Diagram transitions:

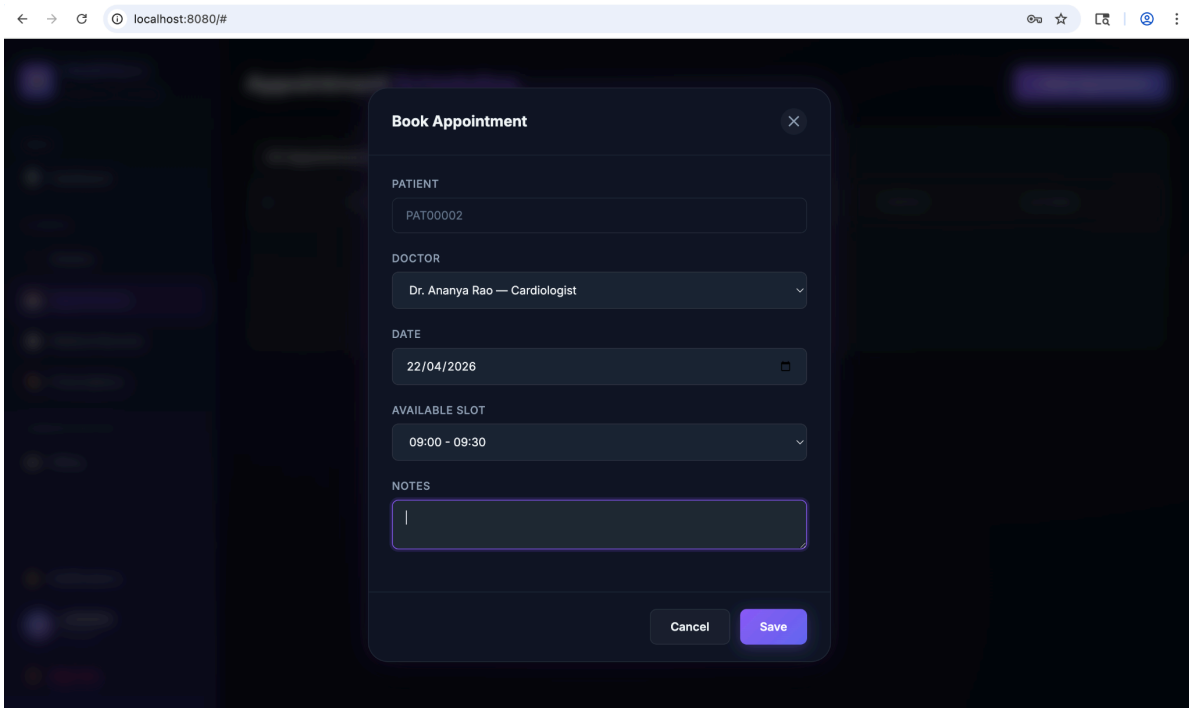
| State Transition (from State Diagram)                  | Subject                | Observer Event Fired                          |
|--------------------------------------------------------|------------------------|-----------------------------------------------|
| Appointment: PendingConfirmation → Scheduled           | AppointmentService     | APPOINTMENT_REMINDER: 'Appointment Confirmed' |
| Appointment: Scheduled → Cancelled                     | AppointmentController  | APPOINTMENT_REMINDER: 'Appointment Cancelled' |
| BillingInsurance: BillingInProgress → InvoiceGenerated | BillingFacade          | BILLING: 'Invoice Generated'                  |
| BillingInsurance: Paid → ReceiptSent                   | BillingFacade          | BILLING: 'Payment Received — Receipt'         |
| LabTest: IN_PROGRESS → COMPLETED                       | LabTestController      | LAB_REPORT: 'Lab Result Ready'                |
| Prescription: PENDING → DISPENSED                      | PrescriptionController | PRESCRIPTION: 'Prescription Dispensed'        |

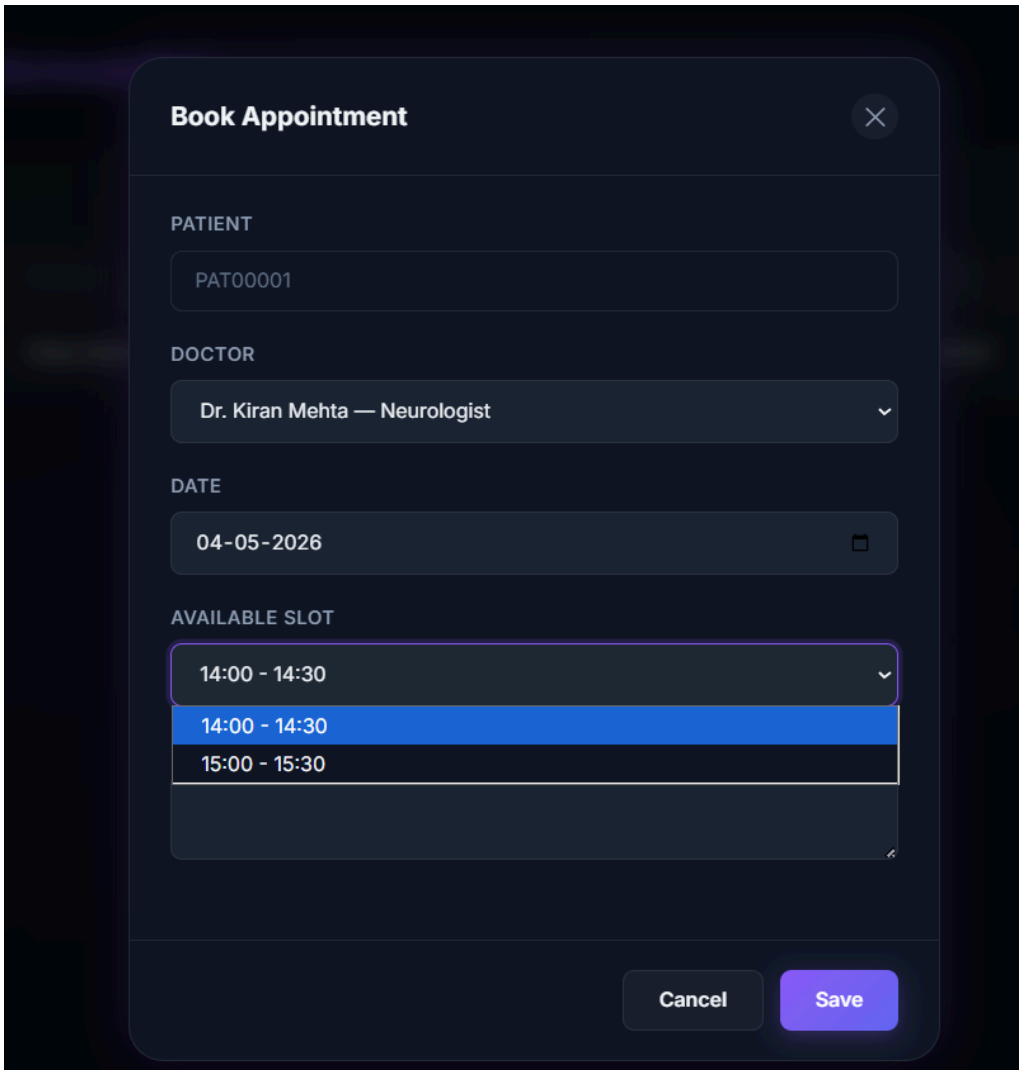
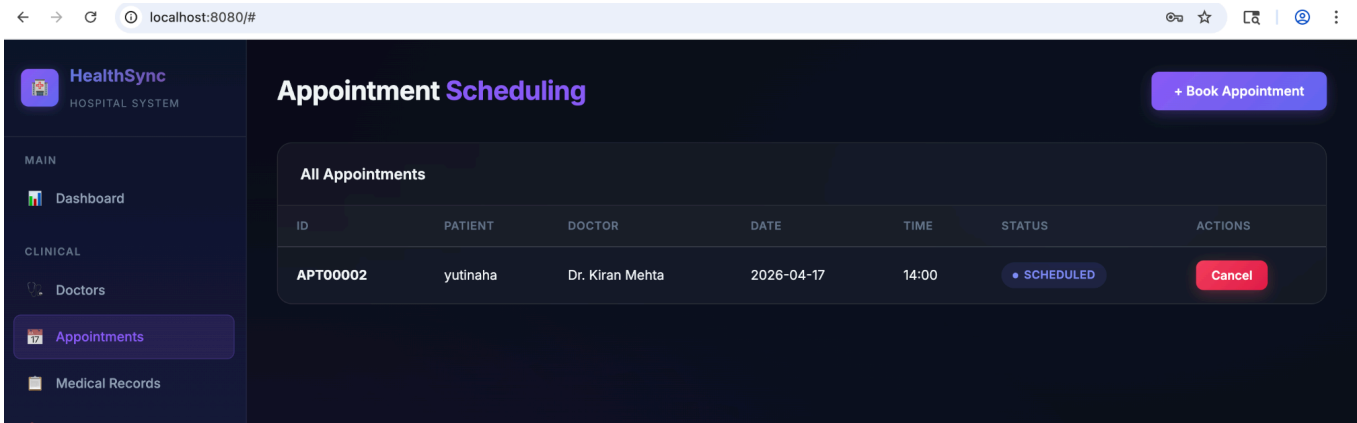
# Section 9 — UI Screenshots

## UC1

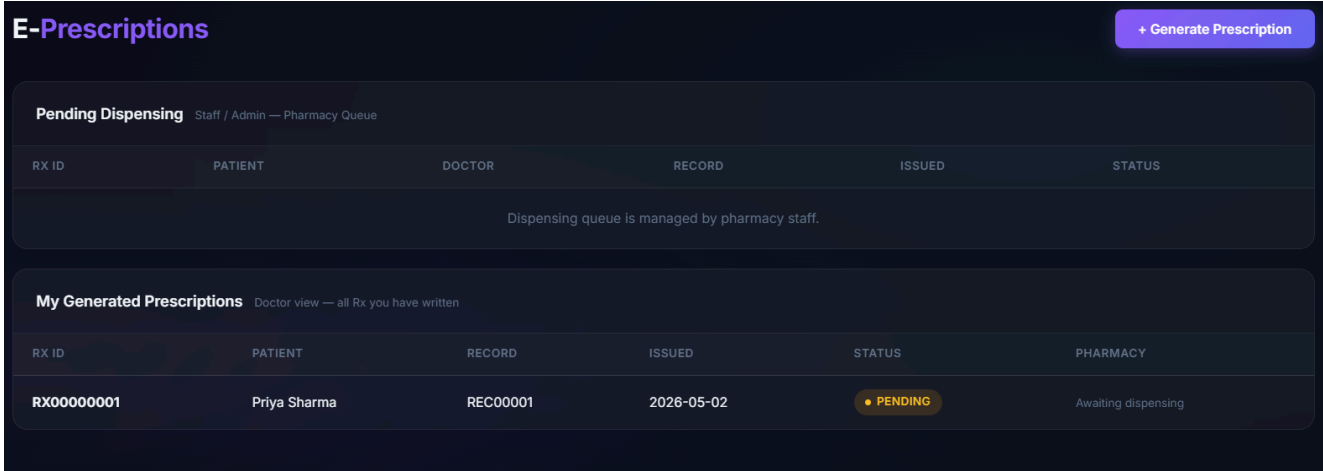
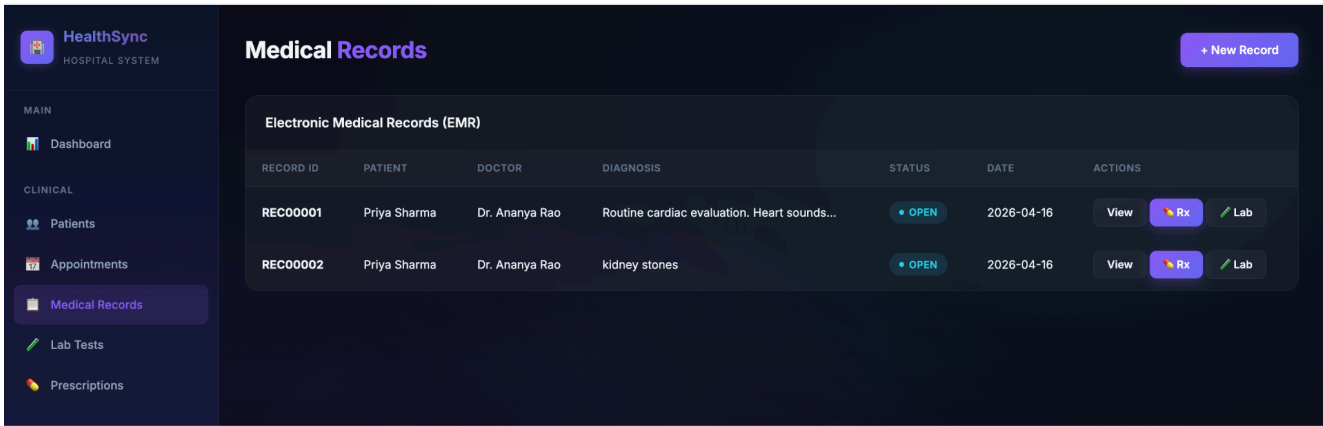
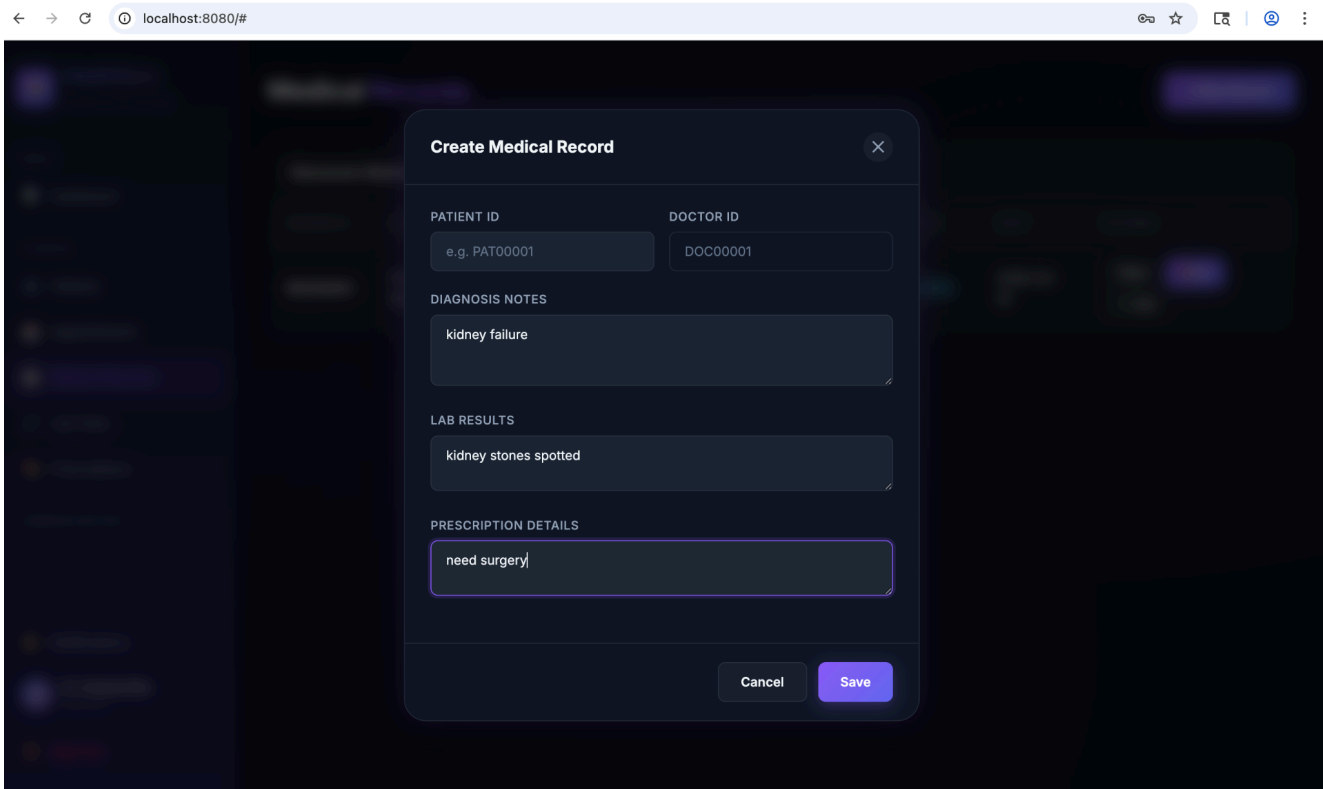


## UC2





UC3



UC4

Generate Invoice

PATIENT ID

PAT00001

CONSULTATION FEE

800

MEDICATION COST

10000

LAB CHARGES

3000

ROOM CHARGES

400

TAX AMOUNT

300

Cancel

Save

Process Payment

BILL AMOUNT (₹)

14500

PAYMENT METHOD

Insurance

INSURANCE PROVIDER

birlaopus

POLICY NUMBER

345678

COVERAGE (₹)

97000

Cancel

Save



Billing & Insurance

All Invoices

| BILL ID  | PATIENT      | TOTAL (₹) | STATUS | DATE       | ACTIONS |
|----------|--------------|-----------|--------|------------|---------|
| BILL0001 | Priya Sharma | ₹1006.25  | PAID   | 2026-05-02 |         |

DATABASE

jdbc:h2:mem:healthsync

+

 APPPOINTMENTS

+

 AUDIT\_LOG

+

 DEPARTMENTS

+

 DOCTORS

+

 DOCTOR\_AVAILABILITY\_SL

+

 INVOICES

+

 LAB\_TESTS

+

 MEDICAL\_RECORDS

+

 NOTIFICATIONS

+

 PATIENTS

+

 PAYMENTS

+

 PHARMACY\_INVENTORY

+

 PRESCRIPTIONS

+

 PRESCRIPTION\_ITEMS

+

 STAFF

+

 STAFF\_ATTENDANCE

+

 USERS

+

 WARDS

+

 WARD\_ALLOTMENTS

+

 INFORMATION\_SCHEMA

+

 Users

+

 H2 2.2.224 (2023-09-17)

Run

Run Selected

Auto complete

Clear

SQL statement:

SELECT \* FROM PAYMENTS;

| AMOUNT_PAID | PAYMENT_ID | PAYMENT_DATE               | BILL_ID  | REFERENCE_NO | PAYMENT_METHOD |
|-------------|------------|----------------------------|----------|--------------|----------------|
| 1006.25     | 1          | 2026-05-02 09:41:17.237645 | BILL0001 | priya@okaxis | UPI            |

(1 row, 1 ms)

Edit

LAB TEST

— Select Test —

Complete Blood Count (CBC)

Blood Glucose (Fasting)

Blood Glucose (Post-Prandial)

HbA1c

Lipid Profile

Liver Function Test (LFT)

Kidney Function Test (KFT)

Thyroid Profile (TSH)

Urine Routine & Microscopy

Chest X-Ray

ECG / EEG

Echocardiogram

MRI Brain

CT Scan

Other (custom)

— Select Test —

Cancel

Save

| Pending Lab Tests Queue |              |                           |                |              |         |               |
|-------------------------|--------------|---------------------------|----------------|--------------|---------|---------------|
| #                       | PATIENT      | TEST NAME                 | ORDERED BY     | ORDERED DATE | STATUS  | ACTIONS       |
| #1                      | Priya Sharma | Liver Function Test (LFT) | Dr. Ananya Rao | 2026-05-02   | ORDERED | Enter Results |

Enter Lab Results

RESULT SUMMARY

Bile production is reduced; liver detoxification is required.

Patient will be auto-notified (FR-06)

Cancel

Save

CLINICAL

Doctors

Appointments

Medical Records

Prescriptions

ADMINISTRATION

Billing

Notifications

Priya Sharma

PATIENT

Notifications

Mark all read

Lab Results Ready — Liver Function Test (LFT)

Your lab test 'Liver Function Test (LFT)' results are now available. Please consult your doctor.

2026-05-02 09:52

E-Prescription Ready

Dr. Dr. Ananya Rao has generated a prescription for you (Ref: RX00000001). It has been sent to the pharmacy.

2026-05-02 09:46

Payment Received — Receipt

Payment of ₹1006.25 received for bill BILL0001 via UPI. Thank you!

2026-05-02 09:41

Appointment Confirmed

INVENTORY

Pharmacy & Inventory

+ Add Medicine

Medicine Inventory

| ID       | MEDICINE          | GENERIC NAME  | STOCK | PRICE (₹) | EXPIRY     | STATUS    | ACTIONS |
|----------|-------------------|---------------|-------|-----------|------------|-----------|---------|
| MED00001 | Paracetamol 500mg | Acetaminophen | 200   | ₹2.5      | 2027-01-01 | In Stock  | Restock |
| MED00002 | Amoxicillin 250mg | Amoxicillin   | 150   | ₹8        | 2026-06-01 | In Stock  | Restock |
| MED00003 | Aspirin 75mg      | Aspirin       | 300   | ₹1        | 2027-03-01 | In Stock  | Restock |
| MED00004 | Metformin 500mg   | Metformin     | 5     | ₹3.5      | 2027-06-01 | Low Stock | Restock |
| MED00005 | Atorvastatin 10mg | Atorvastatin  | 8     | ₹5        | 2027-09-01 | Low Stock | Restock |

Appendix —

| Item                      | Details                                                                      |
|---------------------------|------------------------------------------------------------------------------|
| App URL                   | http://localhost:8080                                                        |
| H2 Console                | http://localhost:8080/h2-console (JDBC: jdbc:h2:mem:healthsync, no password) |
| Source Root               | src/main/java/com/healthsync/                                                |
| Model classes             | model/ — 15+ @Entity JPA classes                                             |
| Controllers               | controller/ — 11 @RestController classes                                     |
| Repositories              | repository/ — 13 @Repository interfaces                                      |
| Factory (Creational #1)   | factory/HealthSyncFactory.java                                               |
| Singleton (Creational #2) | singleton/AuditLogService.java                                               |
| Facade (Structural)       | facade/BillingFacade.java                                                    |
| Observer (Behavioral)     | observer/NotificationObserver.java + PersistentNotificationObserver.java     |
| Frontend (View)           | src/main/resources/static/index.html                                         |
| Seed Data                 | src/main/resources/data.sql                                                  |
| Run command               | mvn spring-boot:run (from HealthSync/ directory)                             |